FEDERAL AUTONOMOUS EDUCATIONAL
INSTITUTION OF HIGHER PROFESSIONAL EDUCATION
«NATIONAL RESEARCH UNIVERSITY
«HIGHER SCHOOL OF ECONOMICS»

Faculty of Informatics, Mathematics, and Computer Science

Novikova Irina Alekseevna

DEVELOPMENT OF A NEW METHOD FOR LLM FINE-TUNING BASED ON IMPORTANCE OF BUILDING BLOCKS

Diploma

in the field of 01.04.02 Applied Mathematics and Informatics

educational program

"Data Mining"

Academic Supervisor
PhD in Computer Science
A.V. Demidovskij

Nizhny Novgorod, 2026

Contents

Introduction

The **relevance** of this work is determined by the pivotal role that Large Language Models (LLMs) currently play in various domains including machine learning [85], computer vision [89], and natural language processing [8]. These pre-trained LLMs are actively employed in numerous industrial applications, prized for their proficiency in tasks such as in-context learning [8], instruction following [69], and structured reasoning [88]. However, the capabilities of a pre-trained LLM may not be sufficient to tackle more specialized tasks, for example, in medical or scientific domains. This necessitates extensive fine-tuning to tailor the models for specific downstream tasks. However, the fine-tuning process is extremely resource-consuming and demands significant memory and time resources, resulting in low training throughput [61]. This way, the evolvement of novel fine-tuning strategies that reduce time and memory costs without sacrificing model quality is of great importance.

At the same time, given the vast parameter space of modern LLMs, which results in a significant over-parameterization [16], identifying and leveraging the most essential model structures becomes crucial. These structures are vital as they disproportionately impact model performance [93]. Effective utilization of LLMs in various scenarios – whether pre-training, fine-tuning, or inference – demands a nuanced understanding of "importance" to discern these critical structures. This requirement extends across multiple optimization strategies, including Neural Architecture Search [19] (NAS), [82], [48], Parameter-Efficient Fine-Tuning (PEFT) [96], [70], quantization [53], [17], [42], and model pruning [25], [77], [81]. Each of these strategies aims to optimize different aspects of model architecture, yet they often rely on disparate heuristics for estimating importance, many of which lack general applicability or direct correlation with model quality [19].

This division highlights a glaring gap: the absence of a general methodology capable of assessing the importance of substructures across different LLM applications. Addressing this gap is not only crucial for enhancing model efficiency but also

for advancing the methodologies used in model optimization, in particular, in the field of model fine-tuning acceleration.

The **practical significance** of this research is that it introduces an innovative fine-tuning strategy for LLMs, which employs both importance and convergence data to enhance the fine-tuning process.

The **novelty** of this work can be summarized as follows:

- This work introduces a comprehensive overview of the most prominent existing methodologies for importance and convergence estimation of different structures within the LLMs.
- This study introduces a new fine-tuning approach for LLMs, that dynamically assigns *LoRA* adapters during the fine-tuning based on importance and convergence analysis.

The **goal** of the present study is to accelerate the fine-tuning process while preserving the LLM quality by introducing a novel *LoRA*-based fine-tuning approach.

To achieve the goal, the following **objectives** must be accomplished:

1. Conduct a comprehensive background study on the topic of fine-tuning acceleration techniques.
2. Provide an overview of existing methodologies for estimating the importance of various structures within LLM and approaches to analyze the convergence of LLMs.
3. Propose a novel *LoRA*-based fine-tuning strategy.
4. Offer a detailed evaluation of the proposed method and its hyperparameters among various LLMs.
5. Conduct a comparative analysis between the proposed solution and the closest competitors in the field.

6. Analyze obtained results and draw conclusion.

The study employs a combination of **methodological approaches** to address the research objectives. Firstly, it adopts a method of theoretical analysis and synthesis of existing literature to establish a solid theoretical foundation for the research. Secondly, programming methods are utilized to analyze the importance and convergence of LLMs'. Lastly, the study employs Deep Learning techniques to fine-tune LLMs.

The **object** of this study is the fine-tuning acceleration of LLMs through importance analysis of Transformer blocks. The **subject** of the study is the feasibility of developing an approach to accelerate the fine-tuning of LLMs through the analysis of importance and convergence of Transformer blocks.

This study was conducted as part of a **research project at the Huawei** dedicated to the fine-tuning acceleration of LLMs.

Chapter 1. Theoretical foundations of fine-tuning acceleration through importance analysis

Nowadays LLMs have recently become the primary choice for various natural language processing scenarios. Unfortunately, the performance of a generic pre-trained model is usually not enough for a specific downstream task. Therefore, it is required to adapt a model for a new domain, which is called fine-tuning [56].

Despite significantly improving LLMs performance on different scenarios, full fine-tuning of LLMs is an extremely resource-intensive process for several compelling reasons. Firstly, every year, the size of LLM grows, leading to a rise in computational costs and hardware requirements. For instance, the LLaMA2-7B model has approximately 100 times fewer trainable parameters compared to the DeepSeek-V3 model. Secondly, since 2016, there has been an increase in the amortized cost of computing by about 2.4 times per year [15] with a prediction to exceed 1 billion by 2027. For instance, the fine-tuning of the GPT-3 model, released in 2020, on a large dataset, necessitates over 3.5 million GPU hours [52] and the full fine-tuning of the LLaMA2-7B model demands more than 140GB of GPU memory [59], [97]. Finally, the environmental factor is significant too. For example, LLaMA3.1-405B model, released in the summer of 2024, requires 25.3 million watts, consuming over 5,000 times more power than the original Transformer [64]. Consequently, the demands for extensive time and considerable memory resources result in a low training throughput and impose a significant financial burden, which is often unsustainable for both corporations and independent researchers. Therefore, the development of innovative training strategies that aim to minimize fine-tuning time and memory costs, while maintaining high-quality output, is of crucial importance.

1.1 Definition of Large Language Model

Let $\mathcal{M}_\Phi$ denote a Transformer autoregressive model with trainable parameters $\Phi \in \mathbb{R}^n$. The model $\mathcal{M}_\Phi$ is described with a set of Transformer blocks τ, where each block τ_i contains two submodules: a multi-head attention module (MHA) (1) and a module consisting of a Feed Forward Neural Network (FFNN) (2). Given the input sequence $x \in \mathbb{R}^n$, MHA module works with h parallel heads.

$$\begin{aligned} head_i &= Softmax(\frac{xW_q(xW_k)^T}{\sqrt{d_h}})xW_v \\ MHA(X) &= Concat(head_1, \ldots, head_h)W_o, \end{aligned} \tag{1}$$

where $i \in \mathbb{R}$, $W_o \in \mathbb{R}^{d \times d}$ is an output projection, and $W_q, W_k, W_v \in \mathbb{R}^{d \times d_h}$ are query, key, and value projections of the i-th head; d_h is typically set to d/h. FFNN module consists of two linear transformations with a ReLU activation.

$$FFNN(x) = ReLU(xW_{up} + b_{up})W_{down} + b_{down}, \tag{2}$$

where $W_{up} \in \mathbb{R}^{d \times d}$ is the up-projection matrix and $W_{down} \in \mathbb{R}^{d \times d}$ is the down-projection matrix.

1.2 Key directions in Large Language Models fine-tuning acceleration

1.2.1 Definition of the fine-tuning process

Fine-tuning is the process of turning a general-purpose model into a specialized one while updating all pre-trained model parameters over a downstream task dataset. Let fine-tuning data be organized as a dataset $D = \{(x_i, y_i), i = 1, \ldots, N\}$, where x_i is a data sample, and y_i is a ground truth label; both x_i, y_i are sequences of tokens. Model prediction is denoted as $\mathcal{M}_\Phi(x_i)$. Fine-tuning assumes that the model is pre-trained: let it be initialized with pre-trained weights Φ_0.

Full fine-tuning is the process of updating pre-trained weights to $\Phi_0 + \Delta\Phi$ so that the fine-tuning objective for $\mathcal{M}_\Phi$ is optimized over Φ (3).

$$\max_{\Phi} \sum_{(x,y)\in D} \sum_{t=1}^{|y|} log(\mathcal{M}_\Phi(y_t|x, y_{<t})), \tag{3}$$

where $|y|$ is the number of tokens in th data sample, dimensionality of $|\Delta\Phi|$ is equal to $|\Phi_0|$ [34].

Various modern approaches exist for reducing time and memory overhead during the full fine-tuning, including model pruning [13], data reduction [21], backpropagation elimination [62], [33], quantization [18], and Parameter-Efficient Fine-Tuning [34].

1.2.2 Model pruning

Model pruning is a technique aimed at reducing the model by eliminating redundant parameters or substructures. This process is primarily categorized into two approaches: structured pruning and unstructured pruning [13].

Structured pruning involves the removal of entire substructures within the neural network, such as filters, channels, Attention heads, or Transformer blocks [70], [50]. On the other hand, unstructured pruning targets individual parameters within the neural network, setting them to zero [98], [27]. For example, N:M pruning approach, which performs unstructured pruning of N weights for every continuous M weights, allows to achieve an acceleration of 47% and memory reduction of 48% compared to the original dense LLM [99]. Despite the benefits of model pruning, several significant drawbacks are associated with current pruning methodologies. These include a notable degradation in the model's quality, as the removal of parameters can lead to a loss of a critical information necessary for accurate predictions [12], [28]. Additionally, the process often introduces computational overhead due to the need for additional inference on calibration data to estimate the importance of different structures within a model [77]. Finally, the efficient implementation of pruned models

can sometimes require specialized hardware, further complicating their deployment and usage in practical applications [99].

In conclusion, while model pruning techniques provide substantial acceleration and memory reduction, they lead to a quality drop, bring time overhead on additional inference on calibration data, and demand special hardware for efficient computation.

1.2.3 Data reduction

Data reduction is the process of selectively removing less significant samples from a training dataset while maintaining the accuracy of the resultant model. This process encompasses three primary methodologies: dataset condensation or distillation, coreset selection, and importance sampling, each defined by its approach to create a compact version of the training dataset.

Dataset distillation involves synthesizing new samples based on statistical properties derived from the original dataset [9]. This method aims to condense the dataset into a smaller, yet representative, set of synthetic samples which effectively capture the underlying data distribution. Coreset selection facilitates the automatic identification and extraction of a subset from the original dataset [35]. This approach ensures that the core elements necessary for learning are retained, thereby optimizing the training process. Importance sampling focuses on selecting samples that contribute most significantly to the model's learning, particularly those that induce larger updates to the model's weights during training [36]. For example, ALOE [84] removes data samples based on losses obtained from a currently trained model or a pre-trained one, providing up to 92% of acceleration with quality degradation of 1.5% compared with *LoRA*. However, these methods cause an additional overhead to the training process by either requiring a full forward pass for all samples in a dataset [37] or training a separate network in parallel [38].

Overall, data reduction techniques provide significant acceleration by reducing the size of the training dataset, lowering the computational resources required for

model training. However, these methods introduce extra overhead during the training process.

1.2.4 Backpropagation elimination

Backpropagation elimination involves replacing the traditional backpropagation method with alternative learning rules that are less computationally expensive. The primary motivation behind this is to achieve significant reductions in both memory and time requirements. This scientific field is represented by two main directions: zeroth-order methods and localized learning strategies.

Zeroth-order methods estimate gradients using only two forward passes. A notable advancement in this category is the Memory-efficient Zeroth-order Optimizer (MeZO) [62], that efficiently implements the ZO-SGD method [79]. It has been shown to deliver performance on par with traditional backpropagation fine-tuning across various tasks. Notably, MeZO achieves up to a twelvefold reduction in memory use and up to a twofold decrease in training time. However, it requires considerably more iterations than conventional fine-tuning and only reaches acceptable levels of accuracy when combined with a specifically tailored prompt [62].

The second category involves localized learning rules, particularly the Hebbian rule. The Hebbian rule [7] draws inspiration from the structural and functional organization of the human brain. It suggests that the strength of synaptic connections between neurons increases when the neurons activate simultaneously and weakens when they do not.

Despite its theoretical appeal, the primary limitation of the Hebbian rule is that its effectiveness is predominantly observed in the training of Convolutional Neural Networks [47], where its combination with gradient-based methods provides an acceleration of up to 1.53x [45]. Moreover,, it has demonstrated a reasonable degree of convergence when applied to large-scale datasets, such as ImageNet [47].

In summary, backpropagation elimination methods significantly reduce computa-

tional demands by lowering memory and training time, but often require more iterations to achieve satisfactory accuracy or show promise only in specific applications such as CNN.

1.2.5 Quantization

Quantization is a technique used to transform high-preci-sion data into low-precision data. This process is predominantly utilized to accelerate inference, but recent studies have also explored its application in enhancing the efficiency of the fine-tuning.

One notable approach is QLoRA [18], which introduces a 4-bit NormalFloat data type for storing frozen pre-trained language model while maintaining computational operations, such as matrix multiplication of *LoRA* adapters, in 16-bit BrainFloat precision. This method achieves a significant acceleration up to 92% with only a minimal 1% degradation in model performance compared to the standard *LoRA* [34]. Another method, PEQA [41], focuses on fine-tuning the quantization scales of quantized Large Language Models, while keeping the integer matrices unchanged. This approach results in up to 92% memory reduction with a relatively small quality degradation of 5% compared to *LoRA*. Additionally, AlphaTuning is a technique that involves quantizing a pre-trained LLM and fine-tuning certain quantized parameters for specific tasks [46]. This method not only accelerates fine-tuning by 38% but also reduces memory usage by 80% with a quality decrease of less than 5%.

In conclusion, while quantization strategies significantly reduce both the time and memory required for fine-tuning, they generally result in a noticeable decline in model quality.

1.2.6 Parameter-Efficient Fine-Tuning

Among all the aforementioned approaches, PEFT emerges as a leading method for accelerating the fine-tuning of LLMs. PEFT focuses on updating a minimal subset of

the model's parameters, significantly reducing time and memory costs, but preserving or even enhancing model accuracy [29]. In the context of PEFT, $\Delta\Phi$ is encoded by a much smaller-sized set of parameters Θ, so that $|\Theta| \ll |\Phi_0|$, $\Delta\Phi = \Delta\Phi(\Theta)$. Thus, PEFT aims at maximizing the fine-tuning objective over Θ (4).

$$max_{\Theta} \sum_{(x,y)\in D} \sum_{t=1}^{|y|} log(\mathcal{M}_{\Phi_0+\Delta\Phi(\Theta)}(y_t|x, y_{<t})) \tag{4}$$

LoRA [34] is the cutting-edge approach in this field, that freezes the pre-trained model weights and injects trainable rank decomposition matrices into each layer of the Transformer architecture, greatly decreasing the number of trainable parameters for downstream tasks. Let $W_0 \in \mathbb{R}^{d_1\times d_2}$ denote pre-trained weights of an arbitrary layer, x is an input data. Then, inference of this layer, denoted as h can be expressed as $h = W_0x$. Applying *LoRA* to that layer modifies its forward pass as follows (5).

$$h = W_0x + \Delta W_x = W_0x + BAx, \tag{5}$$

where $W_0, \Delta W \in \mathbb{R}^{d_1\times d_2}, A \in \mathbb{R}^{r\times d_2}, B \in \mathbb{R}^{d_1\times r}, r \ll (d_1, d_2)$. A, B matrices are called *LoRA* adapters.

By greatly decreasing the number of model parameters, *LoRA* provides up to 50% memory reduction and 150% acceleration compared to full fine-tuning. Although *LoRA* is a state-of-the-art approach, it still has several drawbacks. First, *LoRA* lacks the nuanced capability for more subtle weight adjustment during the fine-tuning [55]. Second, *LoRA* adapters' initialization approach could lead to a slower model convergence [66]. Third, *LoRA* overlooks the varying importance of different weight matrices [96]. Therefore, DoRA [55], PiSSA [66], and AdaLoRA [96] are deemed promising improvements of *LoRA*.

DoRA [55] approach suggests decomposing the pre-trained weights W_0 into *magnitude* and *direction* components (6). *Magnitude* is a trainable vector $M = \|W_0\|_c$, which represents a vector-wise norm of a matrix across each column. *Direction* is the ratio of the pre-trained weight matrix to its norm. The *direction* component is

updated using *LoRA* [34].

$$W = M\frac{W_0 + BA}{\|W_0 + BA\|_c} \tag{6}$$

DoRA provides up to 3% of quality improvement on the fine-tuning of LLaMA2-7B and LLaMA2-13B compared with *LoRA*.

PiSSA [66] approach modifies the *LoRA* by initializing A and B adapter matrices with the principal components of the original matrix W computed utilizing Singular Value Decomposition. Thus, trainable adapters contain the most essential directions of W, that is why training adapters mirror the process of the entire model fine-tuning. *PiSSA* provides up to 21.05% of quality improvement on LLaMA2-7B fine-tuning compared with *LoRA*.

AdaLoRA [96] approach adaptively allocates the parameter budget among the weight matrices according to their importance score. It re-parametrizes the incremental updates in the form of Singular Value Decomposition and modifies the *LoRA* forward pass as follows (7).

$$h = W_0 x + \Delta W_x = W_0 x + P\Lambda Q, \tag{7}$$

where $P \in \mathbb{R}^{d_1 \times r}$ and $Q \in \mathbb{R}^{r \times d_2}$ represent singular vectors of ΔW, the diagonal matrix $\Lambda \in \mathbb{R}^{r \times r}$ contains the singular values $\{\lambda_i\}_{1 \le i \le r}$. *AdaLoRA* provides 1% of quality improvement on DeBERTa-V3-base fine-tuning compared with *LoRA*.

Finally, *LoRA* manually attaches the adapters to all Transformer blocks by a user-defined mask.

Therefore, there exist various approaches for fine-tuning acceleration, with the most prominent being Parameter-Efficient Fine-Tuning. Its state-of-the-art *LoRA* method provides considerable acceleration of up to 150% with memory reduction of 50% maintaining the quality comparable to full fine-tuning. However, *LoRA* manually attaches adapters to all Transformer blocks, causing time and memory overhead. Thus, significant optimization opportunities are presented by selectively attaching

adapters to a subset of the important Transformer blocks.

1.3 Existing approaches for assessing the importance within Large Language Models

1.3.1 Definition of the importance

The concept of the "importance" of certain structures within an LLM primarily arises from the field of model pruning, as discussed earlier. In this context, "importance" facilitates the identification of structures that can be removed without affecting model performance. Thus, understanding the importance of a structure helps assess its impact on the model's overall quality. Similar to model pruning, the evaluation of importance can be implemented for various structures, ranging from individual weights and layers to Transformer Blocks, and even entire model architectures.

1.3.2 Importance of separate weights

The Lottery Ticket Hypothesis [22] suggests that within a randomly-initialized, dense neural network exists a smaller subnetwork, or the *winning ticket* which can achieve the same level of test accuracy as the full network if trained independently, often within the same or even fewer training iterations. This phenomenon has been observed in a range of architectures within both the Computer Vision and Natural Language Processing fields. For instance, a subnetwork within the LeNet architecture, constituting only 51.3% of the original network's weights, has demonstrated the ability to surpass the full model in terms of test accuracy more rapidly. Similarly, the *winning ticket* for the BERT-base model, albeit with a sparsity ratio of 90%, can perform within one standard deviation of the full model's performance [11].

Developing the idea of the importance of individual weights within the model, the concept of systematic outliers in LLMs was introduced, which are defined as values that deviate significantly from the average of their distribution [2]. These outliers

influence the model to predominantly predict the most frequent tokens, thereby affecting the overall quality of the model. Notably, the emergence of these outliers occurs in the Multi-Head Attention module of the Transformer block, significantly influenced by the SoftMax operation (8).

$$A_{ij} = \text{Softmax}\left(\frac{Q_i K_j^T}{\sqrt{d_h}}\right) = \frac{\exp\left(\frac{Q_i K_j^T}{\sqrt{d_h}}\right)}{\sum_{k=1}^{n} \exp\left(\frac{Q_i K_k^T}{\sqrt{d_h}}\right)} \tag{8}$$

$$\text{Let } M = \frac{Q_i K_j^T}{\sqrt{d_h}}, \frac{Q_i K_k^T}{\sqrt{d_h}} = 0, \text{then} \frac{Q_i K_j^T}{\sqrt{d_h}} \gg \frac{Q_i K_k^T}{\sqrt{d_h}},$$

where Q, K are query and key projection matrices, d_h is typically set to d/h, A_{ij} is the attention score.

$$A_{xj} = \frac{\exp(M)}{\exp(M) + (n-1)} \approx 1 \tag{9}$$

$$A_{xk} = \frac{1}{\exp(M) + (n-1)} \approx 0, \tag{10}$$

where n is the number of elements.

This results in dominant attention weights increasing exponentially for key values (9), while other weights approach zero (10). This imbalance in attention weights results in significant updates of the projection weights K and V during backpropagation. If the MHA module produces extreme values due to attention outliers, as the output from the MHA module passes through the MLP block, these anomalies are further amplified by the non-linear activation inside the MLP module and concentrated in the down-projection matrix. Thus, the combination of steep gradients and non-linear transformations fosters the emergence of outliers in both activations and weights.

There were various types of outliers [2] suggested, such as:

1. **Activation outliers inside layer**. For layer outputs $h_l \in \mathbb{R}^{B \times H}$ with B being the batch size, H being the hidden dimension, and $\mu_h = \frac{1}{B \cdot H} \sum_{i,j} |h_{i,j}|$ being

the mean absolute value of layer outputs h_l, the set of activation outliers inside the Transformer block is defined as follows (11).

$$O = \{(i, j) || h_{i,j} > \tau \cdot \mu_h|\}, \tag{11}$$

where τ is a predefined threshold.

2. **Activation outliers inside the down-projection weight**. For down projection in-puts $x_l^{down} \in \mathbb{R}^{B \times H}$ and $\mu_{x^{down}} = \frac{1}{B \cdot H} \sum_{i,j} |x_{i,j}^{down}|$, the set of activation outliers is defined as follows (12).

$$O = \{(i, j) || x_{i,j}^{down}| > \tau \cdot \mu_{x^{down}}\} \tag{12}$$

3. **Weight outliers**. For projection weights $W \in \mathbb{R}^{O \times I}$ with O being output dimension, I being input dimension, and $\mu_{w_i} = \frac{1}{I} \sum_j |w_{i,j}|$ being the row-wise mean absolute value of W, the set of weight outliers is defined as follows (13).

$$O = \{(i, j) || w_{i,j}| > \tau \cdot \mu_{w_i}\} \tag{13}$$

4. **Attention outliers**. For cumulative attention scores $A \in \mathbb{R}^{L \times L}$ with L being a sequence length, $\hat{A}_j = \sum_{i=1}^{L} A_{i,j}$ being the cumulative attention contribution for token j, and $\mu_A = \frac{1}{L} \sum_j \hat{A}_j$, the set of attention outliers is defined as follows (14).

$$O = \{j | \hat{A}_j > \tau \cdot \mu_A\} \tag{14}$$

These systematic outliers significantly impact model quality and pose challenges for model pruning, compression, and quantization [2].

The pruning of critically important weights defined with activations can have a devastating effect on model performance, leading to a quality degradation of up to 50% for LLaMA2-7B in few-shot learning tasks [93]. Conversely, preserving these crucial weights while eliminating the less important ones results in minimal quality degradation, not exceeding 2% [93]. However, even preserving these critical weights

can still lead to quality degradation. The magnitude pruning approach (15) [27], which removes smallest absolute values weights, sometimes results in suboptimal pruning and a degradation in model performance of 15% for the LLaMA2-7B model [81].

$$i_\xi = |W|, \tag{15}$$

where $|W|$ is the absolute value of weight matrix.

Even more advanced methodologies such as Wanda [81] (16), which enhances spar-sity by considering both weight magnitudes and input activations are still facing a similar problem. Despite demonstrating a 10% perplexity improvement for the LLaMA2-7B model compared to the traditional magnitude pruning, Wanda still leads to 5-15% of quality drop compared with the original dense model.

$$i_T = |W| \cdot \|X_T\|_2, \tag{16}$$

where X is the input to the layer.

Therefore, although the methods used to identify significant weights are advantageous for refining the pruning process, they still require further enhancement to preserve the performance of the original LLM.

In terms of quantization, several strategies have been developed to account for the significance of these weights. Outlier-aware quantization, for instance, has demonstrated superior performance, exceeding the state-of-the-art SmoothQuant [90] method by 1% on both the LLaMA2-7B and LLaMA2-13B models [93]. Moreover, skipping the quantization of these extremely important weights and maintaining them in the FP16 data format can enhance model quality by as much as 67% compared to their full quantization [53]. Additionally, sensitivity-based non-uniform quantization, which optimizes bit precision assignments and stores outliers and sensitive weight values in a space-efficient sparse format, results in less than 2% quality degradation when transitioning from a 16-bit to a 4-bit format [43].

Furthermore, the identification of outliers has proven beneficial during the pre-training phase of LLMs. Disabling these outliers during training can lead to a signifi-

cant 60% reduction in model quality compared to training models that include these out-liers [44].

In conclusion, identification and preservation of extremely important weights are crucial for maintaining the efficiency of models during various scenarios. However, this field still requires the development of novel unified methodologies for estimating the importance of separate weights to maintain the original model quality unchanged.

1.3.3 Importance of layers

Moving forward, the significance of various layers within the Transformer model has been extensively studied, revealing intricate details about their functionalities and relative importance.

Previously, experiments on re-initialization and re-randomization robustness of Transformer layers inside LLMs were conducted [95]. This implies re-initializing or assigning random parameters for different model layers, respectively, while keeping the rest of the model parameters intact. As a result, it was revealed that layers within these networks can be classified as either robust or critical: robust layers, when reset to their initial values, do not significantly impact the model's performance. This suggests that not all layers are equally important, and some, especially in over-capacitated networks trained with stochastic gradient descent, are inherently less critical due to the network's self-restricting complexity [95].

Besides, it was observed that individual Attention heads within the Transformer model serve distinct roles in the generation process [87]. Specifically, certain Attention heads are pivotal for identifying syntactic and dependency relationships, highlighting the specialized functions of these components within the architecture. This research also demonstrated that the encoder retains its efficacy despite a considerable reduction in the number of Attention heads, indicating a potential for optimization in the model design [87].

In a related vein, it was noted that perplexity – a measure of model performance

– is more closely associated with overall model capacity than merely the presence of self-attention mechanisms [54]. This finding suggests that increasing the complexity of the model could potentially be more beneficial than focusing solely on the inclusion of self-attention layers.

Further expanding on the theme of layer importance, Tyukin et al. [86] reported that reducing the number of MHA layers by 40% leads to only a minimal decrease in performance. In contrast, a similar reduction in MLP layers results in a significantly greater decline in output quality of up to 60% for the LLaMA2-7B model. This disparity underscores the critical role of MLP layers in maintaining the overall effectiveness of the Transformer model.

Adding to the discourse, He et al. [31] discovered a high level of redundancy within MHA layers, suggesting that a substantial portion of these layers could be removed without notably impairing the model's performance. This finding points towards potential efficiencies in model architecture by minimizing unnecessary layers.

Lastly, Tian et al. [83] elucidated the intricate relationship between attention logits and MLP weights, proposing that explicit updates to self-attention may be redundant. This is due to the implicit incorporation of self-attention functionalities in the lower MLP layers, further emphasizing the possibility of streamlining Transformer architectures without sacrificing performance.

Thus, the investigation into the importance of layers within LLMs constitutes a rapidly evolving field that generates a considerable volume of research. These studies indicate that the layers within LLMs have varying impacts on the final model quality. Specifically, according to research, the Multi-Head Attention module ap-pears to be somewhat redundant, whereas the Multi-Layer Perceptron critically influences the quality of the model's performance.

1.3.4 Importance of Transformer blocks

Moving on to larger structures within LLMs, the saliency of the whole Transformer blocks could be estimated.

[65] corroborates the notion of redundancy in Transformer blocks within LLMs, particularly noting that the redundancy is more pronounced in the middle and later layers, with the initial and final layers being more critical. This differential importance across Transformer blocks implies varying contributions to the model's effectiveness, with the final Transformer block being particularly significant.

Various methodologies have been proposed to assess the importance of Transformer blocks in LLMs. Inspired by the aforementioned Wanda [81] approach, *OWS* [51] selecti-vely prunes Transformer blocks based on their outlier distribution, stating that outliers mostly presented in the first and the last Transformer blocks. *OWS* allows to achieve a 12% reduction in memory usage and a modest 3% quality improvement over *LoRA*. However, *OWS*'s extensive parameters updates make it time-inefficient.

UIDL [25] states that layer-to-layer evolution of representations for a Transformer is given by a residual iteration equation (17).

$$x^{(l+1)} = x^{(l)} + f(x^{(l)}, \theta^{(l)}), \tag{17}$$

where $x^{(l)}$ is the multi-dimensional input, $\theta^{(l)}$ is parameters vector for layer l, $f(x, \theta)$ describes the transformation of one Transformer block.

To successfully prune the n layers before layer l it would be better if the input to the pruned block will be very similar to the output of the pruned block (18).

$$x^{(l)} = x^{(l-n)} + \varepsilon, \tag{18}$$

where ε is a constant indicating the degree to which one input may differ from another.

Therefore, the least important Transformer blocks could be defined as the least affecting on the model output and identified with cosine similarity between input and output to the Transformer block (19).

$$i_T = argmin \frac{1}{\pi} arccos\left(\frac{X_T \cdot Y_T}{\|X_T\|_2 \cdot \|Y_T\|_2}\right), \tag{19}$$

where X_T and Y_T are the input and output to the Transformer block, respectively, $\|\|_2$ is a Frobenius norm.

The proposed approach allows to achieve a 40% reduction in layers without a quality drop for the LLaMA2-70B model [25]. However, *UIDL* requires additional inference on calibration data, which increases time overhead. Besides, cosine similarity was efficiently utilized in ShortGPT [65] leading to 18% of acceleration with quality drop of 14%.

Another approach is *SLEB* [77], which utilizes perplexity as a metric to evaluate Transformer block importance based on the assumption, that redundant blocks contribute less to the model's outputs, and their removal leads to smaller degradation in perplexity.

$$i_T = exp\{-\frac{1}{S}\sum log p_\theta^n(X_T^{(S)}|X_{<T}^{(S)})\}, \tag{20}$$

where X_T is the input to Transformer block, S is a sequence length.

SLEB allows achieving 27% of acceleration with 8% of quality drop on LLaMA2-7B compared to the original model [77]. This method also requires extra inference on calibration data, leading to higher time costs.

Shortened-LLaMA [39] measures the significance of Transformer blocks by their impact on loss, assessing the error caused by the removal of a weight parameter (21).

$$i_T = L(W_T + \Delta W_T) - L(W_T) = \nabla_W L \Delta W_T, \tag{21}$$

where W_T is a Transformer block weight, L is a loss.

Shortened-LLaMA allows achieving 23% of acceleration with 7% of quality drop on LLaMA2-7B compared to the original model [39]. Notably, like *SLEB*, *Shortened-LLaMA* incurs additional time overhead due to the need for inference on the calibration dataset.

In conclusion, the observations made about the importance of Transformer blocks align with those noted in other considered substructures. Although current techniques have demonstrated the ability to achieve a 27% improvement in time and memory efficiency, they still lack a universally effective methodology and lead to considerable

quality degradation of more than 20%. Therefore, it is crucial to further refine and develop importance methods, that enhance model performance without compromising the quality.

1.3.5 Importance of predicted architectures

Finally, the importance of the full architectures could be estimated. Such approaches are being developed within the field of Neural Architecture Search. Neural Architecture Search (NAS) is a technique for automating the design of neural networks. Instead of manually developing neural network architectures, NAS algorithms explore and discover optimal architectures for specific tasks.

For this purpose NAS utilizes a special controller to estimate the performance of a predicted neural architecture on a proxy dataset. However, this evaluation method, particularly when based on validation accuracy, can be computationally prohibitive. For example, NASnet approach requires more than 2000 GPU days to identify optimal model architecture [72].

In response to the high computational demands of traditional NAS methods, zero-cost proxies present a more resource-efficient alternative. Zero-cost methods rely heavily on importance heuristics to approximate the performance of neural architectures without the need for extensive training. For example, the TF-TAS [100] approach utilizes synaptic diversity (22) for the MHA module and synaptic saliency (23) for the FFNN module. TF-TAS improves upon existing techniques, leading to a 4% quality improve-ment compared with manually designed ResNet and MobileNet-V3.

$$i = \|\frac{\partial \mathcal{L}}{\partial W}\| \odot \|W\|_{nuc}, \tag{22}$$

where $\mathcal{L}$ is a loss, W is a weight matrix of a layer inside Transformer, $\|\|_{nuc}$ is the nuclear norm of a matrix.

$$i = \frac{\partial \mathcal{L}}{\partial W} \odot W \tag{23}$$

At the same time, TF-TAS significantly reduces the number of GPU days required to estimate the predicted architecture compared with AutoFormer [10] and ViTAS [80].

A comprehensive comparison study of 15 zero-cost proxies applicable to Transformer architectures an LLMs [19] revealed varying levels of its effectiveness. These proxies were assessed through Kendall and Spearman correlations, with values ranging from -1 to 1, where higher values indicate better predictive accuracy of neural architec-ture rankings vis-à-vis fully trained models. Despite their utility, many proxies exhibi-ted low correlation values, with Kendall's ranging from 0.01 to 0.5 and Spear-man's from 0.1 to 0.5 [19].

Therefore, overview of observed methods is presented in Table 1 and Table 2. analysis of the current methods for evaluating the importance of various structures within LLMs has highlighted several challenges:

1. **Lack of a Unified Methodology**. There is no standard method that applies across different scenarios, making many heuristics inapplicable or unrepresentative in other contexts. Moreover, many existing solutions are completely ineffective in assessing the importance of structures within the LLM.

2. **Quality Degradation**. While existing solutions minimize computational costs, they often lead to a quality reduction from at least 1-2% to complete destruction of LLMs' language modelling capabilities.

3. **Computational Costs**. Most importance-based methods require extensive additional inference on calibration data, which increases time overhead.

As a step towards addressing these challenges and the problem of optimizing the costs of fine-tuning of LLMs, this work will introduce a novel fine-tuning strategy that dynamically allocates *LoRA* adapters to Transformer blocks based on efficient importance analysis to estimate the Transformer blocks suitable for freezing to further accelerate the fine-tuning process. The importance analysis doesn't introduce any additional time overhead, while efficient fine-tuning allows for improving LLM quality after freezing certain units within the LLM.

Table 1: Summary of considered approaches for importance estimation of Transformer blocks and predicted architectures.

Estimated structure	Article	Quality Improvement, %	Ref
Weight	*The Lottery Ticket Hypothesis: Finding Sparse, Trainable Neural Networks*	1.7	[22]
	The Lottery Ticket Hypothesis for Pre-trained BERT Networks	0.8	[11]
	The Super Weight in Large Language Models	-2.4	[93]
	Learning both Weights and Connections for Efficient Neural Networks	-15.4	[28]
	A Simple and Effective Pruning Approach for Large Language Models	-5.5	[81]
	AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration	-2.9	[53]
	SqueezeLLM: Dense-and-Sparse Quantization	2.2	[42]
	BERT Busters: Outlier Dimensions that Disrupt Transformers	1.1	[44]
Layer	*Are all layers created equal?*	-19.5	[95]
	Analyzing Multi-Head Self-Attention: Specialized Heads Do the Heavy Lifting, the Rest Can Be Pruned	-2.4	[87]
	Pay attention to MLPs	5.3	[54]
	Attention Is All You Need But You Don't Need All Of It For Inference of Large Language Models	4.1	[86]
	What Matters in Transformers? Not All Attention is Needed	-5.7	[31]
	JoMA: Demystifying Multilayer Transformers via Joint Dynamics of MLP and Attention	N/A	[83]

1.4 Existing approaches for assessing the convergence within Large Language Models

1.4.1 Definition of the convergence

Convergence of the LLM is the process of achieving the certain optimal by updating model parameters. During training, when the loss function reaches approximately the

Table 2: Summary of considered approaches for importance estimation of Transformer blocks and predicted architectures.

Estimated structure	Article	Acceleration	Quality	Ref
Transformer block	*ShortGPT: Layers in Large Language Models are More Redundant Than You Expect*	18.5	-14.1	[65]
	Outlier-weighed Layerwise Sampling for LLM Fine-tuning	12.3	3.4	[51]
	The Unreasonable Ineffectiveness of the Deeper Layers	N/A	-6.7	[25]
	SLEB: Streamlining LLMs through Redundancy Verification and Elimination of Transformer Blocks	27.6	-8.5	[77]
	Shortened LLaMA: Depth Pruning for Large Language Models with Comparison of Retraining Methods	23.3	-7.4	[39]
Architecture	*Training-free Transformer Architecture Search*	N/A	4.2	[100]
	LPZero: Language Model Zero-cost Proxy Search from Zero	N/A	1.5	[19]

optimum, neural network parameters stop actively changing [1].

1.4.2 Convergence of Transformer blocks

There are various approaches to understanding the convergence of neural networks. To begin with, the cosine similarity [20] could be applied based on the assumption that as the loss function reaches the minimum point, an angle between the weights of the model decreases, indicating that the weights have been trained and since then hardly changed. Another approach is to use the Frobenius norm [1], [3] to estimate the difference between model weights on different training steps. Finally, gradient values [57] can be used to get the update rate of model weights, therefore determining converged Transformer blocks as changing the slowest.

Definition 1 (Convergence metrics)**.** *Let* ∇ *denote the accumulated gradient, T denote the training interval,* $\|\cdot\|_F$ *is a Frobenius norm, then convergence metrics can be*

defined as:

1. *Cosine Similarity [20]:*

$$\hat{c}(W_\xi) = \frac{W_{\xi(T)} \cdot W_{\xi(T-1)}}{\|W_{\xi(T)}\|_2 \cdot \|W_{\xi(T-1)}\|_2} \tag{24}$$

2. *Frobenius Norm [3]*

$$\hat{c}(W_\xi) = \|W_{\xi(T)} - W_{\xi(T-1)}\|_F \tag{25}$$

3. *Gradient Norm [57]*

$$\hat{c}(W_\xi) = \frac{|\|\nabla W_{\xi(T)}\|_2 - \|\nabla W_{\xi(T-1)}\|_2|}{\|\nabla W_{\xi(T-1)}\|_2} \tag{26}$$

Therefore, the convergence analysis of Transformer blocks could be applied to accelerate the fine-tuning process, and blocks, that are closer to convergence are suitable for freezing.

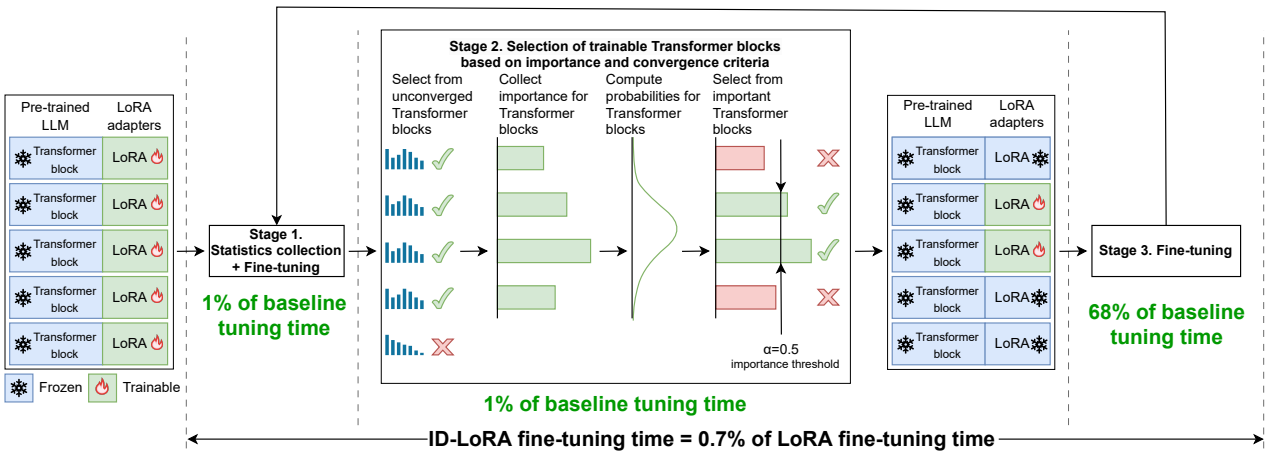

Figure 1: Key idea of proposed fine-tuning method: *Importance-driven LoRA*.

Chapter 2. Development of an approach to accelerate the fine-tuning through importance analysis

The principal components of the proposed method *ID-LoRA* (*Importance-driven LoRA*) are:

1. selective *LoRA* adapters assignment based on Transformer blocks importance,
2. dynamic re-assignment of *LoRA* adapters during the fine-tuning based on tally of importance statistics with subsequent reduction of the number of Transformer blocks,
3. automatic identification of converged layers for further diminution of the number of trainable Transformer blocks.

The *ID-LoRA* method is described in Algorithm 1. Let h denote history defined as the number of steps to collect importance and convergence statistics, α denote the ratio of Transformer blocks to be selected for fine-tuning.

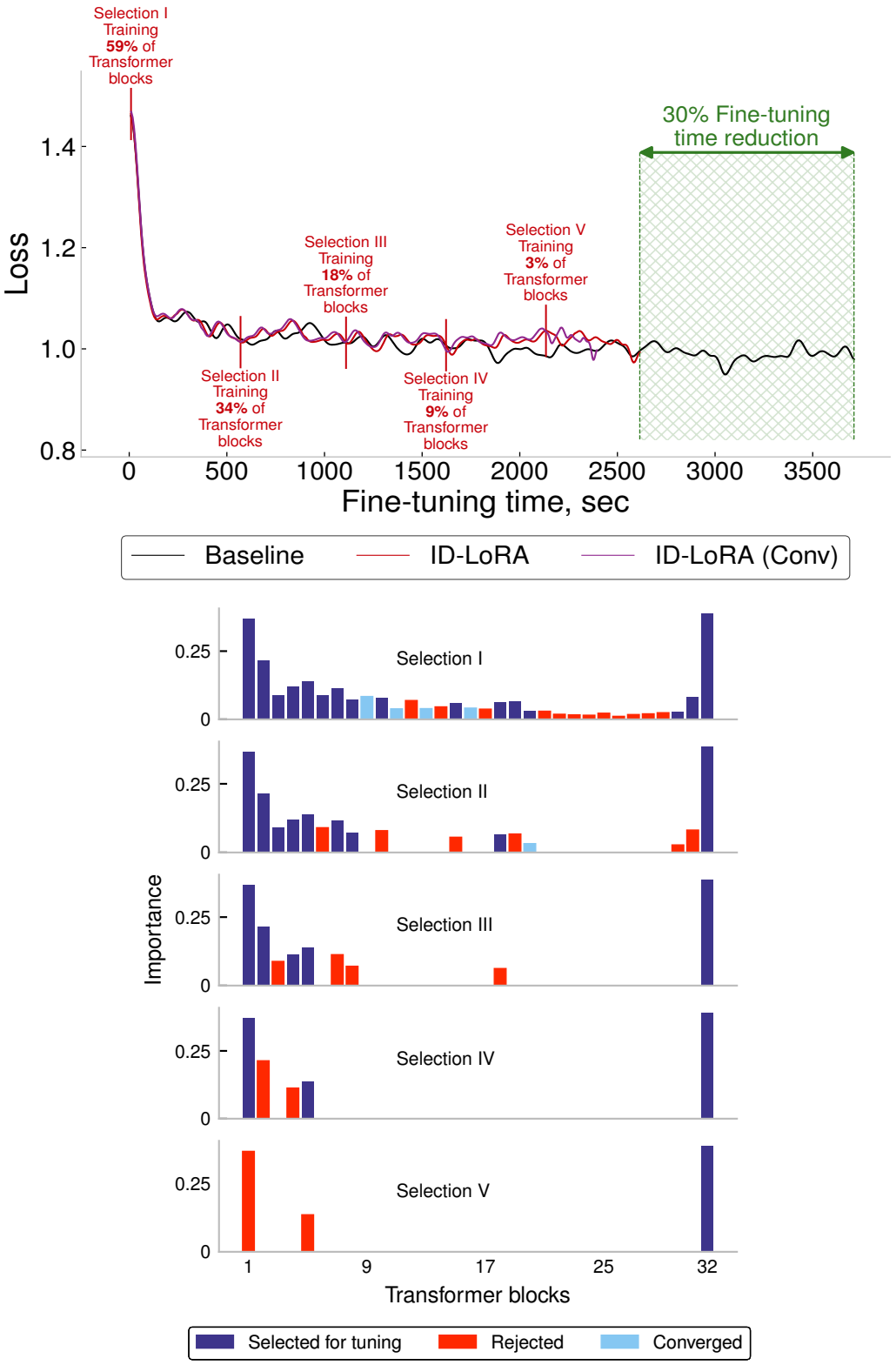

Figure 2: Demonstration of *ID-LoRA* fine-tuning of Qwen2.5-3B model on Alpaca dataset. α is 0.6, h is 10, importance metric is cosine similarity, convergence metric is Gradient norm.

Initially, the fine-tuning process is divided into several periods, during which the algorithm switches the trainable Transformer blocks. Each *ID-LoRA* period has 3 stages: statistics collection, selection of trainable Transformer blocks, and fine-tuning of the selected blocks. At the beginning of each stage, layer-wise importance and, optionally, convergence statistics are collected during h steps. Next, algorithm determines unimportant converged Transformer blocks and removes them from the selection. After that, importance sampling of Transformer blocks is applied, and the ratio of the most important Transformer blocks (α) is selected for further fine-tuning. At the same time, *LoRA* A, B adapters for not-trainable Transformer blocks are merged up to the end of the fine-tuning. Merging non-updatable adapters allows to further accelerate the fine-tuning process and increase memory reduction. Obviously, with each training period, the number of trainable Transformer blocks diminishes. As a result, fine-tuning stops once there are no more trainable Transformer blocks.

Definition 2. *Let T denote Transformer blocks, i_ξ denote importance score for ξ layer inside it, then importance of Transformer block is defined as:*

$$I(T) = \frac{\sum_{\xi \in T} i_\xi}{|T|} \tag{27}$$

2.5 Trainable Transformer Blocks Selection

Statistics Collection The first step of *ID-LoRA* is dedicated to statistics collection over several iterations of fine-tuning. During h steps, importance scores are accumulated for each layer of the model. Since *ID-LoRA* aims to estimate the importance of layers or Transformer blocks within the LLM, only a subset of methods for assessing the importance will be suitable for this task. Among the considered approaches *Magnitude*, *Wanda*, *LISA*, *OWS*, *UIDL*, *SLEB*, and *Shortened-LLaMA* could be appropriate for solving this problem. However, approaches like *SLEB* [77] and *Shortened-LLaMA* [39], which use perplexity and loss-based metrics, are unsuitable for scenarios that require dynamic freezing of non-essential Transformer blocks, as they inhibit gradient calculations. Moreover, these methods necessitate multiple forward passes

Algorithm 1 *ID-LoRA* (α, h)

D: fine-tuning dataset
$\mathcal{M}_\Phi$: pre-trained model
α: trainable blocks selection ratio
h: number of accumulation steps

$C \leftarrow \{\}$: converged Transformer blocks
$Q \leftarrow \{\mathcal{M}_\Phi\}$: trainable Transformer blocks
$I \leftarrow \{\}$: accumulated importance scores
$\mathcal{W} \leftarrow \{\}$: accumulated layer weights
$use\_convergence = True$
for epoch $i = 1, \ldots N$ **do**
  for train step $i = 1, \ldots, K$, global batch $b \in D$ **do**
    $inp, labels \leftarrow b$: data for forward
    if $i < h$ **then**
      {Statistics Collection + Fine-tuning Phase}
      for layer ξ in $\mathcal{M}_\Phi$ **do**
        $inp, out \leftarrow forward(\xi, inp)$
        $I \leftarrow I \cup i(\xi, inp, out)$: calculate layer importance (Definition 3)
        if $use\_convergence$ **then**
          $\mathcal{W} \leftarrow \mathcal{W} \cup w_\xi$
        end if
        $grads \leftarrow backward(loss(out, labels))$
        $apply\_gradients(grads)$
      end for
    else if $i = h$ **then**
      {Selection Phase}
      for $\mathcal{T}$ in Q **do**
        $I_\mathcal{T} \leftarrow I(\{w_j : w_j \in \mathcal{T}\})$: aggregate importance of Transformer blocks (27)
        if $use\_convergence$ **then**
          $C_\mathcal{T} \leftarrow c(\{w_i : w_i \in \mathcal{T}\})$: calculate layer convergence (Definition 1)
        end if
      end for
      $T \leftarrow \{\mathcal{T}_i \in Q : C_{\mathcal{T} \geq \varepsilon}\}$
      $P \leftarrow softmax(\{I_\mathcal{T} : t_i \in T\})$: select with (8)
      $T \leftarrow \alpha * top\_p(Q, P)$: (32)
      $Unfreeze(R)$
      $Freeze(Q \setminus R)$
      $Q \leftarrow R$
    else
      {Fine-tuning Phase}
      $inp, out \leftarrow forward(inp)$
      $grads \leftarrow backward(loss(out, labels))$
      $apply\_gradients(grads)$
    end if
  end for
end for

to evaluate the impact of pruning on validation loss, leading to computational overhead. While *OWS* [51] and *LISA* [70] have shown potential for acceleration without sacrifi-cing model quality, they still incur substantial time and memory costs because they update all parameters of chosen Transformer blocks. Consequently, *Magnitude* [27] and Wanda [81] are identified as the most effective options for overcoming these limitations. In addition, the cosine similarity metric [25] is of interest, as it allows removing Transformer blocks without quality drop. Hence, this study employs these metrics (Definition 3) for importance estimation of Transformer blocks. Notably, Magnitude [27] and Wanda [81] importance rules are adapted to Transformer block importance estimation as described in Definition 2.

Definition 3. *Let W_ξ denote the weight tensor associated with the layer, X_ξ denote the input to the layer, Y_ξ denote the output from the layer, $|\cdot|$ denote the absolute value operator, $\|\cdot\|_2$ is ℓ_2 norm, $W_\mathcal{T}$ denote the weight tensor associated with the Transformer block, $X_\mathcal{T}$ denote the input to the Transformer block, $Y_\mathcal{T}$ denote the output from the Transformer block. Then weight importance metrics can be defined as:*

1. *Magnitude rule [27]:*

$$i(W_\xi, X_\xi, Y_\xi) = |W_\xi| \tag{28}$$

2. *Wanda rule [81]*

$$i(W_\xi, X_\xi, Y_\xi) = |W_\xi| \cdot \|X_\xi\|_2 \tag{29}$$

3. *Cosine Similarity rule [25]*

$$i(W_\mathcal{T}, X_\mathcal{T}, Y_\mathcal{T}) = \frac{1}{\pi}\, arccos\, \left(\frac{X_\mathcal{T} \cdot Y_m athcalT}{\|X_\mathcal{T}\|_2 \cdot \|Y_\mathcal{T}\|_2}\right) \tag{30}$$

$$P(T) = \frac{e^{I(T)}}{\sum_{k=1}^{\mathcal{M}_\Phi} e^{I(k)}}, \tag{31}$$

where I denotes Transformer block importance score.

$$Top_P = \{T_k \in \mathcal{M}_\Phi : \sum_{k=1}^{\mathcal{M}_\Phi} P(T_k) \leq p\}, \tag{32}$$

where p denotes the threshold.

Selection Phase Upon reaching the h number of steps, the importance scores of each Transformer block are averaged. After that, the second step of *ID-LoRA* begins, aimed at selecting important Transformer blocks. Based on the importance scores, selection probability for each Transformer block is computed (31). Next, the Top-P sampling (32) is applied, which samples Transformer blocks with the highest probability scores until the sum of the scores reaches the specified threshold value. Then, the sampling probability for each Transformer block is recalculated (31). Finally, α of the most important Transformer blocks are selected for further fine-tuning.

2.6 Converged Transformer Blocks Selection (Optional)

Besides the importance analysis, another way to estimate the essential Transformer blocks for the fine-tuning is an optional convergence analysis. It identifies fully trained Transformer blocks that could be frozen, thereby further reducing the number of trainable Transformer blocks. Once per h steps, additional statistics for convergence analysis are accumulated.

Definition 4. *Let ε denote convergence threshold, $\hat{c}$ denote convergence metric. Then convergence rule for a layer is defined as:*

$$c(W_\xi) = \begin{cases} 1, & \hat{c}(W_\xi) < \varepsilon \\ 0, & otherwise \end{cases}$$

Definition 5. *Transformer block T is converged if the following inequality with convergence threshold δ is satisfied:*

$$C(T) = \frac{\sum_{\xi \in T} c(W_\xi)}{|T|} > \delta \tag{33}$$

When statistic has been collected, the converged Transformer blocks are identified with convergence (Definition 4) with the help of convergence metrics (Definition 1, 5). After that, converged Transformer blocks are frozen up to the end of training, which leads to further reduction of the number of trainable parameters and an earlier stop of the fine-tuning process.

2.7 Key Differences from Existing Approaches

Firstly, importance-based pruning of Transformer blocks typically leads to considerable quality drop, necessitating further fine-tuning to restore LLM performance after pruning [77], [39], [12]. In contrast, *ID-LoRA* integrates the pruning process with parameter-efficient fine-tuning with *LoRA*, thereby preventing performance degradation and enhancing model quality. Secondly, whereas considered pruning methods (*UIDL*, *Shortened-LLaMA*, *SLEB*) incur additional computational costs by requiring inference on calibration data, *ID-LoRA* identifies essential Transformer blocks efficiently without extra time. Thirdly, in comparison with existing LoRA modifications, *ID-LoRA* offers an innovative approach that enables automatic assignment of *LoRA* A, B adapters only to important Transformer blocks, significantly reducing time without compromising the accuracy. Finally, *ID-LoRA* applies the idea of additional conver-gence analysis to further reduce the number of trainable Transformer blocks.

Chapter 3. Evaluation of the proposed method

To assess the feasibility of the *ID-LoRA*, in this study a comprehensive evaluation of the proposed solution has been applied.

3.8 Models

For the evaluation purposes, six LLMs with different numbers of parameters were chosen to estimate the performance on different scales of LLMs:

- GPT2-M with 24 Transformer blocks;
- LLaMA2-7B with 32 Transformer blocks;
- LLaMA2-13B with 40 Transformer blocks;
- Qwen2.5-3B with 36 Transformer blocks;
- Qwen2.5-7B with 28 Transformer blocks;
- Qwen2.5-32B with 64 Transformer blocks.

The GPT2-M model was taken from *transformers v4.33.2*, LLaMA2-7B, LLaMA2-13B, Qwen2.5-3B, Qwen2.5-7B, Qwen2.5-32B models were taken from HuggingFace hub [5], [4], [74], [75], [73].

3.9 Overview of considered competitors

To validate the efficacy of *ID-LoRA*, a comparative analysis was employed with alternative approaches in the domain of fine-tuning acceleration and importance analysis of LLM layers and Transformer blocks.

Examined methods to analyze the importance of Transformer blocks include:

1. *UIDL* [25] approach. After estimating the most important model layers during the inference on calibration data and further pruning of the least essential structures, *UIDL* utilizes QLoRA fine-tuning to enhance model quality.

2. The *SLEB* [77] approach.

3. The *Shortened-LLaMA* [39] approach.

4. The *OWS* [51] approach. After estimating the importance during the inference on calibration data and further pruning of the least essential structures, *OWS* utilizes full fine-tuning to enhance model quality.

5. The *LISA* [70] approach, which randomly selects Transformer blocks during the process of full fine-tuning. This method does not rely on any deliberate heuristic to estimate the importance, that is why it can be considered as the reference approach for understanding the effect of the "importance".

Notably, all these approaches, except *LISA*, employ different heuristics to estimate Transformer block importance, which could provide insights into the effectiveness of existing metrics and their correlation with LLM performance. Also, it helps to evaluate whether using these importance measures can optimize the fine-tuning process compared to *LoRA*. For practical implementation, the official implementation of con-sidered approaches was taken from corresponding Github repositories: *UIDL* [26], *SLEB* [78], *Shortened-LLaMA* [40], *OWS* [51], *LISA* [70].

Since *LoRA* is the cutting-edge approach for fine-tuning acceleration, it was selected as a baseline for computational experiments on quality and acceleration of considered approaches. In addition, the most prominent modifications of *LoRA*, such as *DoRA* [55], *PiSSA* [66], *AdaLoRA* [96], and *LISA* [70] were chosen as reference methods during the evaluation. The implementation of *LoRA*, *DoRA*, *PiSSA*, and *AdaLoRA* approaches was taken from the *peft v.0.18.0* library [63].

3.10 Training and evaluation protocols

Firstly, for the *UIDL*, *SLEB*, *Shortened-LLaMA*, and *OWS* approaches, the importance estimation of Transformer blocks was conducted on a validation set of the Alpaca dataset. Subsequently, for the *UIDL* the QLoRA fine-tuning was applied to

key Transformer blocks, while non-essential ones were left unchanged. In contrast, for *OWS* the full fine-tuning of critical units was utilized, while others were frozen. Originally, *SLEB* and *Shortened-LLaMA* do not employ fine-tuning to improve model performance. However, following the pruning of non-essential Transformer blocks, which involved their total removal, the *LoRA* fine-tuning method was utilized to important Transformer blocks.

Table 3: Training hardware used for the evaluation of considered approaches.

	GPT2-M	**LLaMA2-7B, LLaMA2-13B, Qwen2.5-3B, Qwen2.5-7B, Qwen2.5-32B**
Processor	CPU 32 cores with 3.0GHz	CPU 32 cores with 2.90GHz
OS	Ubuntu 20.04.6 LTS	Ubuntu 20.04.04 LTS
Driver Version	515.43.04	535.183.01
Kernel Version	5.4.0-176-generic	5.15.0-125-generic
CUDA Toolkit Version	11.7.64	12.2
Training Libraries	*transformers* 4.33.0, *PyTorch* 2.1.0, *DeepSpeed* 0.15.4	*transformers* 4.45.0, *PyTorch* 2.1.0, *DeepSpeed* 0.15.4

Table 4: Training protocol used for the evaluation of considered approaches.

	GPT2-M	**LLaMA2-7B, LLaMA2-13B, Qwen2.5-3B, Qwen2.5-7B, Qwen2.5-32B**
Sequence Length	512	256
Learning Rate	6e-4	1e-5
AdamW β_1	0.9	0.9
AdamW β_2	0.99	0.99
Weight Decay	0.01	0
Epochs	2	2
Precision	fp16	bf16
LoRA r	16	16
LoRA α	32	32
Ratio of Transformer blocks	0.8	0.6

Used training hardware and software are described in Table 3. Training protocol for each model is described in Table 4. The *Ratio of Transformer blocks* row means the ratio of Transformer blocks selected for fine-tuning with chosen approaches. Notably, the GPT2-M model requires a higher number of updated layers than all other models, because a greater number of layers in the model allows for a smaller amount of updated layers, since a substantial proportion of the layers will still be updated.

After the fine-tuning, in order to estimate the final quality of the model, zero-shot accuracy was measured on commonsense reasoning datasets: PIQA [6], HellaSwag [94], Winogrande [76], ARC [14], OpenbookQA [68], MMLU [32] using the lm-evaluation-harness package [23]. Zero-shot PPL was measured on language modelling dataset WikiText2 [67].

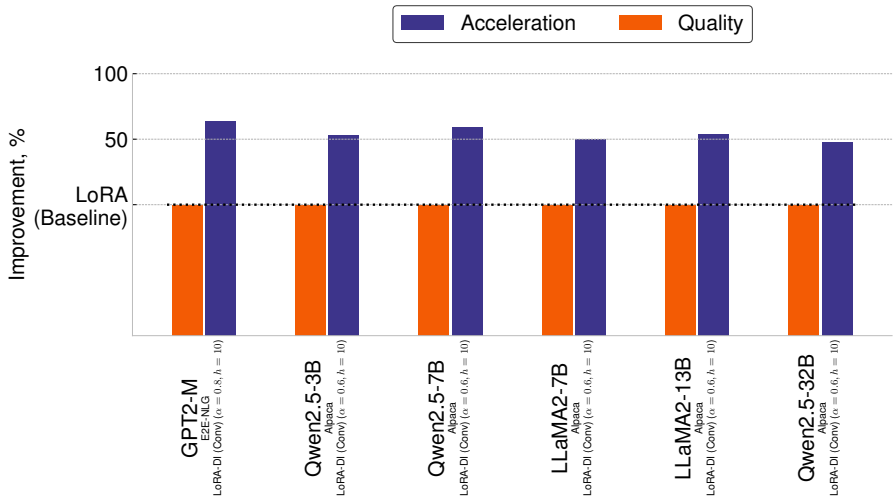

Figure 3: Overall *ID-LoRA* performance. The baseline is *LoRA* fine-tuning marked as dotted black line. Quality is defined as BLEU for GPT2-M, and PPL for other models. Acceleration is computed as the ratio of time required for baseline fine-tuning to the time required for fine-tuning with *ID-LoRA*.

3.11 Analysis of the obtained results

Computational experiments with chosen models and approaches are presented in Table 5, Table 6, Table 7, Table 8. The best results of fine-tuning acceleration are presented in Table 7 and Table 8, while the best results on quality are presented in Table 5 and Table 6, where for *Wikitext* the quality is measured as perplexity, and a lower value is better. For other datasets the quality is measured as accuracy, and a higher is better. It is important to note, that for LLaMA-family and Qwen-family models, the evaluation of *LISA* and *OWS* approaches was not presented, as memory requirements on the full fine-tuning of these models did not fit to used hardware.

ID-LoRA provides an average acceleration of 33.1% with a considerable maximum speedup of 44.8% for LLaMA2-13B with a quality decrease of less than 0.1%. Additional convergence analysis allows *ID-LoRA* to stop training earlier by ensuring the convergence of all Transformer blocks, leading to the increased speedup on average at 54.8% over *ID-LoRA* without convergence analysis. *ID-LoRA (Conv)* provides a greater quality degradation of 0.3% compared to the baseline *LoRA* approach, as some Transformer blocks during the fine-tuning process may not receive essential

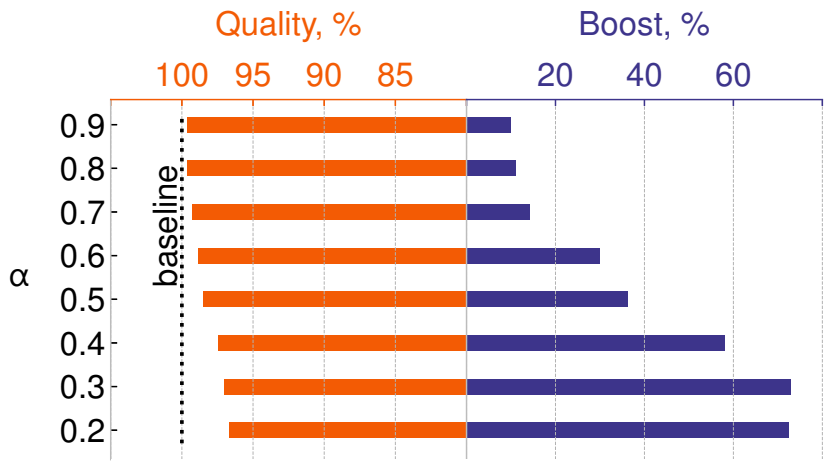

Figure 4: Sensitivity analysis of *ID-LoRA* to the α hyperparameter. h is 10, importance metric is cosine similarity.

information because of the early stop. In addition, *ID-LoRA* significantly outperforms *PiSSA*, *DoRA*, and *AdaLoRA* from the point of acceleration, and provides 0.8% of quality improvement compared to *AdaLoRA*. However, it leads to 0.1% and 1.6% quality decrease compared to *DoRA* and *PiSSA* respectively as *PiSSA* employs SVD initialization of *LoRA* A, B adapters and *DoRA* incorporates additional magnitude vector during the fine-tuning.

Performance analysis of *ID-LoRA* compared to the such state-of-the-art pruning approaches as UIDL, SLEB, and Shortened-LLaMA revealed, that *ID-LoRA* provides 1.1%, 43.8%, and 79.6% quality improvement compared to *UIDL*, *SLEB*, and *Shortened-LLaMA*, respectively, and outperforms all these methods from the point of acceleration.

3.12 Analysis of *ID-LoRA* hyperparameters

Ablation study of the α hyperparameter of *ID-LoRA* is presented in Figure 4. It was analyzed in terms of their impact on the resulting quality and acceleration. Obtained results demonstrates that increasing α significantly affects both acceleration and model quality. In particular, a maximum speedup of 72.6% is observed at α =

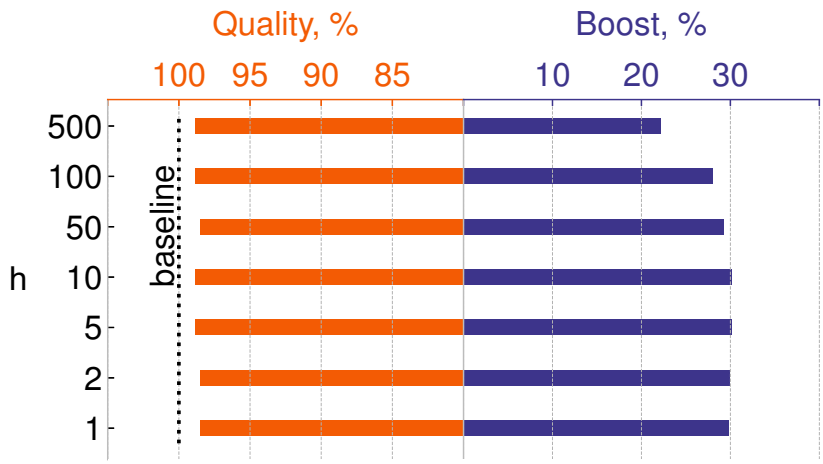

Figure 5: Sensitivity analysis of *ID-LoRA* to the h hyperparameter. α is 0.6, importance metric is cosine similarity.

0.2, where only 20% of the Transformer blocks are trained. However, it leads to a 3.3% increase in perplexity, suggesting underfitting. Minimal quality degradation (0.3%) occurs at higher α values (0.8 and 0.9), where 80-90% of the Transformer blocks are trained initially. Ablation study of the h hyperparameter of *ID-LoRA* is presented in Figure 5. It was analyzed in terms of their impact on the resulting quality and acceleration. According to the achieved results, variations do not significantly impact quality, with only a 0.3% change in perplexity was observed between $h = 1$ and $h = 500$. The smallest hvalue accelerates fine-tuning by 29.8%, indicating that even a single training step can adequately capture the importance of a Transformer block. This observation is supported by the relative stability of model quality across different h settings, as shown in Figure 4b. $h = 10$ is identified as the optimal setting for balancing speed and quality. Ablation study of importance metrics (Definition 3), and convergence metrics (Definition 4) is presented in Figure 8. Investigation into importance metrics (Definition 3), summarized in Figure 6, reveals that the Magnitude metric yields the best balance between acceleration and quality. In contrast, the Wanda metric, despite considering input activations, provides a higher speedup at the cost of increased quality degradation by 2% compared to the baseline. Although cosine similarity decelerates the fine-tuning on 29.6% compared with the Magnitude,

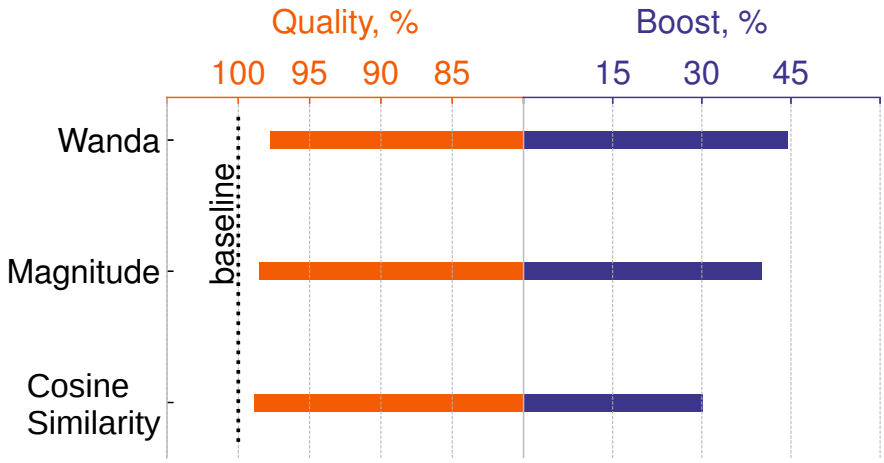

Figure 6: Sensitivity analysis of *ID-LoRA* to considered importance metrics. α is 0.6, h is 10.

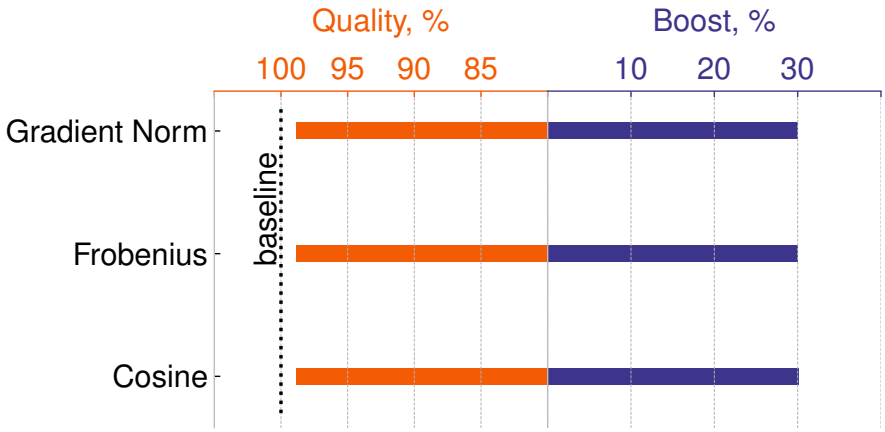

Figure 7: Sensitivity analysis of *ID-LoRA* to considered convergence metrics. α is 0.6, h is 10, importance metric is cosine similarity.

Figure 8: Sensitivity analysis of *ID-LoRA* to considered importance and convergence metrics.

it degrades model quality the least, representing the most suitable choice, as minimizing the drop in model quality is particularly crucial in the context of importance-based fine-tuning. As a result of the convergence metrics analysis depicted in Figure 7, it was found that any of the metrics are good enough for the given fact.

Table 5: Quality results after the fine-tuning with considered approaches on GPT2-M, LLaMA2-7B, and LLaMA2-13B.

Model	Method	ARC-e	Hella-swag	MMLU	Openbook QA	PIQA	Wikitext	Wino-grande
GPT2-M	*LoRA* (baseline)	40.4	30.7	22.9	17.4	63.5	173.1	50.6
	DoRA	39.4	30.3	22.9	16.6	62.2	184.2	50.5
	PiSSA	34.3	28.8	22.9	15.4	57.7	172.1	50.2
	AdaLoRA	37.6	30.3	22.9	17.6	61.7	179.2	51.4
	UIDL	41.0	31.2	**23.0**	17.4	63.1	173.8	51.8
	LISA	**49.1**	**33.3**	22.9	18.6	**67.6**	205.7	**53.1**
	OWS	25.4	25.8	22.9	12.2	52.6	180.2	49.1
	SLEB	38.4	30.2	22.9	14.2	61.5	450.1	50.8
	Shortened-LLaMA	31.4	26.6	22.9	13.2	55.2	400.5	49.4
	ID-LoRA	40.5	31.2	22.9	18.6	62.4	72.7	52.6
	ID-LoRA (Conv)	42.0	31.4	22.9	**19.4**	64.2	**63.4**	50.8
LLaMA2-7B	*LoRA* (baseline)	78.4	57.7	40.8	33.6	78.8	**9.2**	69.8
	DoRA	78.4	57.7	40.8	33.6	78.7	**9.2**	69.8
	PiSSA	**79.2**	**58.6**	**42.0**	**35.2**	**79.2**	9.4	**71.0**
	AdaLoRA	77.1	57.3	41.7	32.4	78.3	**9.2**	69.1
	UIDL	77.7	57.5	38.8	33.4	78.6	**9.2**	69.8
	SLEB	47.7	38.6	26.0	23.6	64.9	403.6	55.3
	Shortened-LLaMA	47.2	35.5	25.2	21.6	65.3	53.1	51.5
	ID-LoRA	78.0	57.6	40.0	33.0	78.5	**9.2**	69.5
	ID-LoRA (Conv)	77.8	57.5	39.9	33.0	78.4	**9.2**	69.5
LLaMA2-13B	*LoRA* (baseline)	80.2	60.4	**53.5**	37.0	79.4	**7.7**	72.2
	DoRA	80.2	60.4	53.4	37.0	79.3	**7.7**	72.0
	PiSSA	**80.9**	**61.3**	**53.5**	**39.2**	**80.0**	7.9	71.9
	AdaLoRA	79.6	60.0	52.3	35.2	79.1	**7.7**	72.4
	UIDL	79.9	60.5	53.1	37.0	79.1	**7.7**	72.0
	SLEB	62.3	46.3	28.6	26.8	72.5	20.0	62.4
	Shortened-LLaMA	46.2	33.5	33.4	25.2	60.0	72.8	56.0
	ID-LoRA	80.2	60.5	52.7	36.6	78.9	**7.7**	**72.5**
	ID-LoRA (Conv)	80.3	60.5	53.1	36.2	78.8	**7.7**	72.3

Table 6: Quality results after the fine-tuning with considered approaches on Qwen2.5-3B, Qwen2.5-7B, and Qwen2.5-32B.

Model	Method	ARC-e	Hella-swag	MMLU	Openbook QA	PIQA	Wikitext	Wino-grande
Qwen2.5-3B	*LoRA* (baseline)	79.1	55.1	65.0	30.8	78.2	15.5	69.4
	DoRA	78.9	55.1	65.0	30.8	78.1	15.3	69.4
	PiSSA	**80.0**	**56.0**	64.6	**32.0**	78.2	**11.8**	**70.2**
	AdaLoRA	78.3	54.7	64.8	29.6	**78.3**	15.1	68.1
	UIDL	78.4	55.2	65.2	31.2	78.2	15.5	68.0
	SLEB	60.5	36.3	24.2	23.8	69.5	77.4	52.5
	Shortened-LLaMA	78.3	54.7	64.8	29.6	**78.3**	15.1	68.1
	ID-LoRA	79.0	55.1	65.0	30.8	78.2	15.5	69.3
	ID-LoRA (Conv)	77.7	54.9	**65.1**	30.2	78.0	14.9	68.3
Qwen2.5-7B	*LoRA* (baseline)	82.6	59.6	71.5	33.0	79.4	8.8	73.3
	DoRA	82.6	59.8	71.7	32.8	79.6	8.8	73.1
	PiSSA	**83.7**	**60.5**	70.5	33.8	**79.8**	9.0	72.1
	AdaLoRA	80.7	59.7	71.9	33.8	78.8	**8.7**	73.3
	UIDL	82.6	59.7	**71.8**	32.6	78.9	8.8	72.5
	SLEB	56.1	35.3	24.9	19.0	65.5	40.3	52.0
	Shortened-LLaMA	33.4	27.9	23.6	14.8	56.9	542.3	51.1
	ID-LoRA	81.5	59.5	71.7	**34.0**	79.2	**8.7**	73.2
	ID-LoRA (Conv)	81.2	59.4	71.4	33.6	79.1	8.8	**73.7**
Qwen2.5-32B	*LoRA* (baseline)	82.2	64.8	80.8	33.8	81.7	5.95	76.6
	PiSSA	**85.2**	**65.4**	79.2	**35.4**	**82.4**	6.1	**77.2**
	AdaLoRA	80.8	64.8	80.7	33.4	82.0	6.0	75.8
	UIDL	82.4	64.8	**81.0**	33.6	81.6	6.0	76.5
	SLEB	72.6	47.9	53.5	27.6	73.7	18.7	61.9
	Shortened-LLaMA	56.6	44.4	40.2	22.4	53.0	250.8	51.6
	ID-LoRA	81.2	64.8	80.6	33.8	81.5	**5.6**	75.8
	ID-LoRA (Conv)	81.3	64.8	80.7	32.6	81.7	**5.6**	76.5

Table 7: Acceleration results after the fine-tuning with considered approaches.

Model	Method	Inference time, s	Fine-tuning time, s	Fine-tuning time acceleration, %	Cumulative acceleration, %
GPT2-M	*LoRA* (baseline)	–	8965.9	–	–
	DoRA	–	10897.8	-21.6	-21.6
	PiSSA	–	8878.5	0.9	0.9
	AdaLoRA	–	9078.96	-1.3	-1.3
	UIDL	8.8	12504.1	-39.5	-39.5
	LISA	–	12635.3	-40.9	-40.9
	OWS	**5.3**	14460.9	-61.3	-61.3
	SLEB	215.8	7560.5	16.7	13.5
	Shortened-LLaMA	268.7	7566.5	16.5	12.6
	ID-LoRA	–	6707.7	25.2	25.2
	ID-LoRA (Conv)	–	**3282.1**	**63.4**	**63.4**
LLaMA2-7B	*LoRA* (baseline)	–	3712.6	–	–
	DoRA	–	15120.2	-407.3	-407.3
	PiSSA	–	3650.7	1.7	1.7
	AdaLoRA	–	5626.7	-51.6	-51.6
	UIDL	**446.7**	4425.1	-19.2	-31.2
	SLEB	1891.1	2229.9	40.0	-11.0
	Shortened-LLaMA	882.4	2337.3	37.1	13.4
	ID-LoRA	–	2627.8	29.2	29.2
	ID-LoRA (Conv)	–	**2643.5**	**50.0**	**50.0**
LLaMA2-13B	*LoRA* (baseline)	–	8408.1	–	–
	DoRA	–	25503.0	-203.3	-203.3
	PiSSA	–	8433.8	-0.3	-0.3
	AdaLoRA	–	16222.0	-92.9	-92.9
	UIDL	**682.2**	9628.2	14.5	-22.7
	SLEB	8907.9	5389.3	34.1	-70.0
	Shortened-LLaMA	12533.3	5183.7	38.3	-110.7
	ID-LoRA	–	4639.3	44.8	44.8
	ID-LoRA (Conv)	–	**3868.8**	**54.0**	**54.0**

Table 8: Acceleration results after the fine-tuning with considered approaches.

Model	Method	Inference time, s	Fine-tuning time, s	Fine-tuning time acceleration, %	Cumulative acceleration, %
Qwen2.5-3B	*LoRA* (baseline)	–	3145.3	–	–
	DoRA	–	9467.4	-201.0	-201.0
	PiSSA	–	3164.5	-0.6	-0.6
	AdaLoRA	–	5393.8	-71.5	-71.5
	UIDL	**304.0**	3474.4	-10.5	-20.1
	SLEB	2265.3	1837.2	41.6	-30.4
	Shortened-LLaMA	1127.4	2013.8	36.1	0.1
	ID-LoRA	–	2246.0	28.6	28.6
	ID-LoRA (Conv)	–	**1474.6**	**53.1**	**53.1**
Qwen2.5-7B	*LoRA* (baseline)	–	3824.9	–	–
	DoRA	–	32945.9	-761.3	-761.3
	PiSSA	–	3797.2	0.7	0.7
	AdaLoRA	–	5453.7	-42.6	-42.6
	UIDL	**277.1**	4741.9	10.5	-31.2
	SLEB	1575.0	2296.1	40.1	-29.7
	Shortened-LLaMA	955.7	2467.5	35.5	10.0
	ID-LoRA	–	2554.0	33.2	33.2
	ourmethodshortconv	–	**1557.8**	**59.3**	**59.3**
Qwen2.5-32B	*LoRA* (baseline)	–	28617.7	–	–
	PiSSA	–	28896.5	-1.0	-1.0
	AdaLoRA	–	55076.2	-92.5	-92.5
	UIDL	**1606.4**	34279.2	-19.8	-25.4
	SLEB	35594.3	17188.3	40.1	-84.4
	Shortened-LLaMA	32814.3	19069.9	33.4	-81.3
	ID-LoRA	–	17875.7	37.5	37.5
	ID-LoRA (Conv)	–	**14950.0**	**48.8**	**48.8**

Conclusion

This study was aimed to address to primary objectives:

- Provide an in-depth examination of existing importance estimation techniques across various scenarios, with a particular focus on the importance estimation of Transformer blocks to optimize the fine-tuning process.

- Introduce a novel fine-tuning approach for LLMs, that dynamically assigns *LoRA* adapters during the fine-tuning based on importance and convergence analysis of Transformer blocks.

Throughout the research, various modern approaches for reducing time and memory overhead during the fine-tuning were considered with LoRA de facto being the state-of-the-art solution in the field. However, it was revealed, that *LoRA* manually attaches low-parameterized adapters to the whole model by a user-defined mask, leading to computational overhead due to the LLMs overparametrization. Moreover, the concept of "importance" was deemed as the identification of structures that can be removed without significantly affecting model performance. It was found, that many of existing importance-based compression techniques leads to considerable quality degradation and provide substantial costs due to additional inference on calibration data.

To address the aforementioned challenges, in the second part of this study, a novel *LoRA*-based fine-tuning approach, *ID-LoRA*, that utilizes importance estimation for efficient reallocation of *LoRA* adapters has been proposed. To thoroughly evaluate the proposed method *ID-LoRA* a performance and comparison analysis on different LLMs and datasets has been applied. As a baseline, a cutting-edge *LoRA* approach has been used. In addition, key Transformer blocks-based importance approaches like *UIDL*, *OWS*, *SLEB*, *LISA*, and *Shortened-LLaMA* and the most prominent modifications of *LoRA* – *DoRA*, *PiSSA*, and *AdaLoRA* – were chosen for reference.

The 6 models' fine-tuning was estimated with considered approaches, including

GPT2-M, LLaMA2-7B, LLaMA2-13B, Qwen2.5-3B, Qwen2.5-7B, and Qwen2.5-32B. After the fine-tuning, in order to estimate the final quality of the model, zero-shot accuracy was measured on commonsense reasoning datasets: PIQA [6], HellaSwag [94], Winogrande [76], ARC [14], OpenbookQA [68], MMLU [32] using the lm-evaluation-harness package [23]. Zero-shot PPL was measured on language modelling dataset WikiText2 [67].

The results showed that:

- Proposed *ID-LoRA* demonstrates a notable acceleration of 53% for Qwen2.5-3B, 59% for Qwen2.5-7B, 50% for LLaMA2-7B, 54% for LLaMA2-13B, and 49% for Qwen2.5-32B without sacrificing the accuracy.
- Proposed *ID-LoRA* significantly outperforms all considered reference importance estimation and fine-tuning acceleration approaches from the point of fine-tuning time acceleration while preserving the quality.

Therefore, future directions of this study within research project in Huawei include:

1. Provide a theoretical justification for the effectiveness of the proposed *ID-LoRA*.
2. Conduct an analysis, whether entirely freezing Transformer blocks during the fine-tuning may be imprudent, since it is possible that temporarily removing them from the fine-tuning with subsequent reactivation could provide quality improvement.
3. Explore the interoperability of *ID-LoRA* with *LoRA* modifications that could lead to mixed results when using *ID-LoRA* with other *LoRA*-based approaches.

References

[1] Z. Allen-Zhu, Y. Li, and Z. Song, "A convergence theory for deep learning via over-parameterization," 2019. [Online]. Available: https://arxiv.org/abs/1811.03962

[2] Y. An, X. Zhao, T. Yu, M. Tang, and J. Wang, "Systematic outliers in large language models," 2025. [Online]. Available: https://arxiv.org/abs/2502.06415

[3] S. Arora, N. Cohen, N. Golowich, and W. Hu, "A convergence analysis of gradient descent for deep linear neural networks," 2019. [Online]. Available: https://arxiv.org/abs/1810.02281

[4] D. Autar, "daryl149/llama-2-13b-chat-hf. Hugging Face — huggingface.co," https://huggingface.co/daryl149/llama-2-13b-chat-hf, 2023, [Accessed 20-04-2025].

[5] ——, "daryl149/llama-2-7b-hf. Hugging Face — huggingface.co," https://huggingface.co/daryl149/llama-2-7b-hf, 2023, [Accessed 20-04-2025].

[6] Y. Bisk, R. Zellers, R. L. Bras, J. Gao, and Y. Choi, "Piqa: Reasoning about physical commonsense in natural language," 2019. [Online]. Available: https://arxiv.org/abs/1911.11641

[7] R. Brown, "Donald o. hebb and the organization of behavior: 17 years in the writing," *Molecular Brain*, vol. 13, 12 2020.

[8] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. M. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray,

B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, and D. Amodei, “Language models are few-shot learners,” 2020. [Online]. Available: https://arxiv.org/abs/2005.14165

[9] G. Cazenavette, T. Wang, A. Torralba, A. A. Efros, and J.-Y. Zhu, “Dataset distillation by matching training trajectories,” 2022. [Online]. Available: https://arxiv.org/abs/2203.11932

[10] M. Chen, H. Peng, J. Fu, and H. Ling, “Autoformer: Searching transformers for visual recognition,” 2021. [Online]. Available: https://arxiv.org/abs/2107.00651

[11] T. Chen, J. Frankle, S. Chang, S. Liu, Y. Zhang, Z. Wang, and M. Carbin, “The lottery ticket hypothesis for pre-trained bert networks,” 2020. [Online]. Available: https://arxiv.org/abs/2007.12223

[12] X. Chen, Y. Hu, J. Zhang, Y. Wang, C. Li, and H. Chen, “Streamlining redundant layers to compress large language models,” 2025. [Online]. Available: https://arxiv.org/abs/2403.19135

[13] H. Cheng, M. Zhang, and J. Q. Shi, “A survey on deep neural network pruning-taxonomy, comparison, analysis, and recommendations,” 2024. [Online]. Available: https://arxiv.org/abs/2308.06767

[14] P. Clark, I. Cowhey, O. Etzioni, T. Khot, A. Sabharwal, C. Schoenick, and O. Tafjord, “Think you have solved question answering? try arc, the ai2 reasoning challenge,” 2018. [Online]. Available: https://arxiv.org/abs/1803.05457

[15] B. Cottier, R. Rahman, L. Fattorini, N. Maslej, T. Besiroglu, and D. Owen, “The rising costs of training frontier ai models,” 2025. [Online]. Available: https://arxiv.org/abs/2405.21015

[16] M. Denil, B. Shakibi, L. Dinh, M. Ranzato, and N. de Freitas, "Predicting parameters in deep learning," 2014. [Online]. Available: https://arxiv.org/abs/1306.0543

[17] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer, "Llm.int8(): 8-bit matrix multiplication for transformers at scale," 2022. [Online]. Available: https://arxiv.org/abs/2208.07339

[18] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "Qlora: Efficient finetuning of quantized llms," 2023. [Online]. Available: https://arxiv.org/abs/2305.14314

[19] P. Dong, L. Li, X. Liu, Z. Tang, X. Liu, Q. Wang, and X. Chu, "Lpzero: Language model zero-cost proxy search from zero," 2024. [Online]. Available: https://arxiv.org/abs/2410.04808

[20] A. Draganov, S. Vadgama, and E. J. Bekkers, "The hidden pitfalls of the cosine similarity loss," 2024. [Online]. Available: https://arxiv.org/abs/2406.16468

[21] U. Endriss, F. S. Melo, K. Bach, A. J. B. Diz, J. M. Alonso-Moral, S. Barro, and F. Heintz, Eds., *ECAI 2024 - 27th European Conference on Artificial Intelligence, 19-24 October 2024, Santiago de Compostela, Spain - Including 13th Conference on Prestigious Applications of Intelligent Systems (PAIS 2024)*, ser. Frontiers in Artificial Intelligence and Applications, vol. 392. IOS Press, 2024. [Online]. Available: https://doi.org/10.3233/FAIA392

[22] J. Frankle and M. Carbin, "The lottery ticket hypothesis: Finding sparse, trainable neural networks," 2019. [Online]. Available: https://arxiv.org/abs/1803.03635

[23] L. Gao, J. Tow, B. Abbasi, S. Biderman, S. Black, A. DiPofi, C. Foster, L. Golding, J. Hsu, A. Le Noac'h, H. Li, K. McDonell, N. Muennighoff, C. Ociepa, J. Phang, L. Reynolds, H. Schoelkopf, A. Skowron, L. Sutawika,

E. Tang, A. Thite, B. Wang, K. Wang, and A. Zou, "A framework for few-shot language model evaluation," 07 2024. [Online]. Available: https://zenodo.org/records/12608602

[24] J. Gou, B. Yu, S. J. Maybank, and D. Tao, "Knowledge distillation: A survey," *International Journal of Computer Vision*, vol. 129, no. 6, p. 1789–1819, Mar. 2021. [Online]. Available: http://dx.doi.org/10.1007/s11263-021-01453-z

[25] A. Gromov, K. Tirumala, H. Shapourian, P. Glorioso, and D. A. Roberts, "The unreasonable ineffectiveness of the deeper layers," 2025. [Online]. Available: https://arxiv.org/abs/2403.17887

[26] ——, "The unreasonable ineffectiveness of the deeper layers," https://github.com/arcee-ai/PruneMe, 2025, [Accessed 22-06-2025].

[27] S. Han, H. Mao, and W. J. Dally, "Deep compression: Compressing deep neural networks with pruning, trained quantization and huffman coding," 2016. [Online]. Available: https://arxiv.org/abs/1510.00149

[28] S. Han, J. Pool, J. Tran, and W. J. Dally, "Learning both weights and connections for efficient neural networks," 2015. [Online]. Available: https://arxiv.org/abs/1506.02626

[29] Z. Han, C. Gao, J. Liu, J. Zhang, and S. Q. Zhang, "Parameter-efficient fine-tuning for large models: A comprehensive survey," 2024. [Online]. Available: https://arxiv.org/abs/2403.14608

[30] J. He, S. Wu, W. Wen, C. J. Xue, and Q. Li, "Chess: Optimizing llm inference via channel-wise thresholding and selective sparsification," 2024. [Online]. Available: https://arxiv.org/abs/2409.01366

[31] S. He, G. Sun, Z. Shen, and A. Li, "What matters in

transformers? not all attention is needed," 2024. [Online]. Available: https://arxiv.org/abs/2406.15786

[32] D. Hendrycks, C. Burns, S. Basart, A. Zou, M. Mazeika, D. Song, and J. Steinhardt, "Measuring massive multitask language understanding," 2021. [Online]. Available: https://arxiv.org/abs/2009.03300

[33] G. Hinton, "The forward-forward algorithm: Some preliminary investigations," 2022. [Online]. Available: https://arxiv.org/abs/2212.13345

[34] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "Lora: Low-rank adaptation of large language models," 2021. [Online]. Available: https://arxiv.org/abs/2106.09685

[35] J. hun Shim, K. Kong, and S.-J. Kang, "Core-set sampling for efficient neural architecture search," 2021. [Online]. Available: https://arxiv.org/abs/2107.06869

[36] A. H. Jiang, D. L. K. Wong, G. Zhou, D. G. Andersen, J. Dean, G. R. Ganger, G. Joshi, M. Kaminksy, M. Kozuch, Z. C. Lipton, and P. Pillai, "Accelerating deep learning by focusing on the biggest losers," 2019. [Online]. Available: https://arxiv.org/abs/1910.00762

[37] T. B. Johnson and C. Guestrin, "Training deep models faster with robust, approximate importance sampling," in *Advances in Neural Information Processing Systems*, S. Bengio, H. Wallach, H. Larochelle, K. Grauman, N. Cesa-Bianchi, and R. Garnett, Eds., vol. 31. Curran Associates, Inc., 2018.

[38] A. Katharopoulos and F. Fleuret, "Biased importance sampling for deep neural network training," 2017. [Online]. Available: https://arxiv.org/abs/1706.00043

[39] B.-K. Kim, G. Kim, T.-H. Kim, T. Castells, S. Choi, J. Shin, and

H.-K. Song, “Shortened llama: Depth pruning for large language models with comparison of retraining methods,” 2024. [Online]. Available: https://arxiv.org/abs/2402.02834

[40] ——, “Shortened llama: Depth pruning for large language models with comparison of retraining methods,” https://github.com/Nota-NetsPresso/shortened-llm, 2024, [Accessed 22-06-2025].

[41] J. Kim, J. H. Lee, S. Kim, J. Park, K. M. Yoo, S. J. Kwon, and D. Lee, “Memory-efficient fine-tuning of compressed large language models via sub-4-bit integer quantization,” 2023. [Online]. Available: https://arxiv.org/abs/2305.14152

[42] S. Kim, C. Hooper, A. Gholami, Z. Dong, X. Li, S. Shen, M. W. Mahoney, and K. Keutzer, “Squeezellm: Dense-and-sparse quantization,” 2024. [Online]. Available: https://arxiv.org/abs/2306.07629

[43] ——, “Squeezellm: Dense-and-sparse quantization,” 2024. [Online]. Available: https://arxiv.org/abs/2306.07629

[44] O. Kovaleva, S. Kulshreshtha, A. Rogers, and A. Rumshisky, “Bert busters: Outlier dimensions that disrupt transformers,” 2021. [Online]. Available: https://arxiv.org/abs/2105.06990

[45] S. Krithivasan, S. Sen, S. Venkataramani, and A. Raghunathan, “Accelerating dnn training through selective localized learning,” *Frontiers in Neuroscience*, vol. 15, 01 2022.

[46] S. J. Kwon, J. Kim, J. Bae, K. M. Yoo, J.-H. Kim, B. Park, B. Kim, J.-W. Ha, N. Sung, and D. Lee, “Alphatuning: Quantization-aware parameter-efficient adaptation of large-scale pre-trained language models,” 2022. [Online]. Available: https://arxiv.org/abs/2210.03858

[47] G. Lagani, F. Falchi, C. Gennaro, and G. Amato, "Hebbian semi-supervised learning in a sample efficiency setting," *Neural Networks*, vol. 143, p. 719–731, Nov. 2021. [Online]. Available: http://dx.doi.org/10.1016/j.neunet.2021.08.003

[48] N. Lee, T. Ajanthan, and P. H. S. Torr, "Snip: Single-shot network pruning based on connection sensitivity," 2019. [Online]. Available: https://arxiv.org/abs/1810.02340

[49] P. Li, L. Yin, X. Gao, and S. Liu, "Outlier-weighed layerwise sampling for llm fine-tuning," https://github.com/pixeli99/OWS, 2024, [Accessed 22-06-2025].

[50] ——, "Outlier-weighed layerwise sampling for llm fine-tuning," 2025. [Online]. Available: https://arxiv.org/abs/2405.18380

[51] ——, "Outlier-weighed layerwise sampling for llm fine-tuning," 2025. [Online]. Available: https://arxiv.org/abs/2405.18380

[52] X. Li, Y. Yao, X. Jiang, X. Fang, X. Meng, S. Fan, P. Han, J. Li, L. Du, B. Qin, Z. Zhang, A. Sun, and Y. Wang, "Flm-101b: An open llm and how to train it with $100k budget," 2025. [Online]. Available: https://arxiv.org/abs/2309.03852

[53] J. Lin, J. Tang, H. Tang, S. Yang, W.-M. Chen, W.-C. Wang, G. Xiao, X. Dang, C. Gan, and S. Han, "Awq: Activation-aware weight quantization for llm compression and acceleration," 2024. [Online]. Available: https://arxiv.org/abs/2306.00978

[54] H. Liu, Z. Dai, D. R. So, and Q. V. Le, "Pay attention to mlps," 2021. [Online]. Available: https://arxiv.org/abs/2105.08050

[55] S.-Y. Liu, C.-Y. Wang, H. Yin, P. Molchanov, Y.-C. F. Wang, K.-T. Cheng,

and M.-H. Chen, “Dora: Weight-decomposed low-rank adaptation,” 2024. [Online]. Available: https://arxiv.org/abs/2402.09353

[56] Y. Liu, A. Singh, C. D. Freeman, J. D. Co-Reyes, and P. J. Liu, “Improving large language model fine-tuning for solving math problems,” 2023. [Online]. Available: https://arxiv.org/abs/2310.10047

[57] Y. Liu, S. Agarwal, and S. Venkataraman, “Autofreeze: Automatically freezing model blocks to accelerate fine-tuning,” 2021. [Online]. Available: https://arxiv.org/abs/2102.01386

[58] Z. Liu, B. Oguz, C. Zhao, E. Chang, P. Stock, Y. Mehdad, Y. Shi, R. Krishnamoorthi, and V. Chandra, “Llm-qat: Data-free quantization aware training for large language models,” 2023. [Online]. Available: https://arxiv.org/abs/2305.17888

[59] K. Lv, Y. Yang, T. Liu, Q. Gao, Q. Guo, and X. Qiu, “Full parameter fine-tuning for large language models with limited resources,” 2024. [Online]. Available: https://arxiv.org/abs/2306.09782

[60] X. Ma, G. Fang, and X. Wang, “Llm-pruner: On the structural pruning of large language models,” 2023. [Online]. Available: https://arxiv.org/abs/2305.11627

[61] S. Malladi, T. Gao, E. Nichani, A. Damian, J. D. Lee, D. Chen, and S. Arora, “Fine-tuning language models with just forward passes,” 2024. [Online]. Available: https://arxiv.org/abs/2305.17333

[62] ——, “Fine-tuning language models with just forward passes,” 2024. [Online]. Available: https://arxiv.org/abs/2305.17333

[63] S. Mangrulkar, S. Gugger, L. Debut, Y. Belkada, S. Paul, and

B. Bossan, “Peft: State-of-the-art parameter-efficient fine-tuning methods,” https://github.com/huggingface/peft, 2022.

[64] N. Maslej, L. Fattorini, R. Perrault, Y. Gil, V. Parli, N. Kariuki, E. Capstick, A. Reuel, E. Brynjolfsson, J. Etchemendy, K. Ligett, T. Lyons, J. Manyika, J. C. Niebles, Y. Shoham, R. Wald, T. Walsh, A. Hamrah, L. Santarlasci, J. B. Lotufo, A. Rome, A. Shi, and S. Oak, “Artificial intelligence index report 2025,” 2025. [Online]. Available: https://arxiv.org/abs/2504.07139

[65] X. Men, M. Xu, Q. Zhang, B. Wang, H. Lin, Y. Lu, X. Han, and W. Chen, “Shortgpt: Layers in large language models are more redundant than you expect,” 2024. [Online]. Available: https://arxiv.org/abs/2403.03853

[66] F. Meng, Z. Wang, and M. Zhang, “Pissa: Principal singular values and singular vectors adaptation of large language models,” 2025. [Online]. Available: https://arxiv.org/abs/2404.02948

[67] S. Merity, C. Xiong, J. Bradbury, and R. Socher, “Pointer sentinel mixture models,” 2016. [Online]. Available: https://arxiv.org/abs/1609.07843

[68] T. Mihaylov, P. Clark, T. Khot, and A. Sabharwal, “Can a suit of armor conduct electricity? a new dataset for open book question answering,” 2018. [Online]. Available: https://arxiv.org/abs/1809.02789

[69] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. L. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, J. Schulman, J. Hilton, F. Kelton, L. Miller, M. Simens, A. Askell, P. Welinder, P. Christiano, J. Leike, and R. Lowe, “Training language models to follow instructions with human feedback,” 2022. [Online]. Available: https://arxiv.org/abs/2203.02155

[70] R. Pan, X. Liu, S. Diao, R. Pi, J. Zhang, C. Han, and T. Zhang, “Lisa: Layerwise importance sampling for memory-efficient large language model fine-tuning,” 2024. [Online]. Available: https://arxiv.org/abs/2403.17919

[71] ——, "Lisa: Layerwise importance sampling for memory-efficient large language model fine-tuning," https://github.com/osehmathias/lisa, 2024, [Accessed 22-06-2025].

[72] X. Qin and Z. Wang, "Nasnet: A neuron attention stage-by-stage net for single image deraining," 2020. [Online]. Available: https://arxiv.org/abs/1912.03151

[73] Qwen, "Qwen/qwen2.5-32b. Hugging Face — huggingface.co," https://huggingface.co/Qwen/Qwen2.5-32B, 2024, [Accessed 20-04-2025].

[74] ——, "Qwen/qwen2.5-3b. Hugging Face — huggingface.co," https://huggingface.co/Qwen/Qwen2.5-3B, 2024, [Accessed 20-04-2025].

[75] ——, "Qwen/qwen2.5-7b. Hugging Face — huggingface.co," https://huggingface.co/Qwen/Qwen2.5-7B, 2024, [Accessed 20-04-2025].

[76] K. Sakaguchi, R. L. Bras, C. Bhagavatula, and Y. Choi, "Winogrande: An adversarial winograd schema challenge at scale," 2019. [Online]. Available: https://arxiv.org/abs/1907.10641

[77] J. Song, K. Oh, T. Kim, H. Kim, Y. Kim, and J.-J. Kim, "Sleb: Streamlining llms through redundancy verification and elimination of transformer blocks," 2024. [Online]. Available: https://arxiv.org/abs/2402.09025

[78] ——, "Sleb: Streamlining llms through redundancy verification and elimination of transformer blocks," https://github.com/jiwonsong-dev/SLEB, 2025, [Accessed 22-06-2025].

[79] J. Spall, "Multivariate stochastic approximation using a simultaneous perturbation gradient approximation," *IEEE Transactions on Automatic Control*, vol. 37, no. 3, pp. 332–341, 1992.

[80] X. Su, S. You, J. Xie, M. Zheng, F. Wang, C. Qian, C. Zhang, X. Wang,

and C. Xu, “Vitas: Vision transformer architecture search,” 2021. [Online]. Available: https://arxiv.org/abs/2106.13700

[81] M. Sun, Z. Liu, A. Bair, and J. Z. Kolter, “A simple and effective pruning approach for large language models,” 2024. [Online]. Available: https://arxiv.org/abs/2306.11695

[82] H. Tanaka, D. Kunin, D. L. K. Yamins, and S. Ganguli, “Pruning neural networks without any data by iteratively conserving synaptic flow,” 2020. [Online]. Available: https://arxiv.org/abs/2006.05467

[83] Y. Tian, Y. Wang, Z. Zhang, B. Chen, and S. Du, “Joma: Demystifying multilayer transformers via joint dynamics of mlp and attention,” 2024. [Online]. Available: https://arxiv.org/abs/2310.00535

[84] A. Tugaryov, A. Trutnev, A. Demidovskij, and I. Salnikov, “Aloe: Boosting large language model fine-tuning with aggressive loss-based elimination of samples,” 10 2024.

[85] T. Tuncer, F. Ertam, S. Dogan, E. Aydemir, and P. Pławiak, “Ensemble residual networks based gender and activity recognition method with signals,” *The Journal of Supercomputing*, vol. 76, p. 2119–2138, 03 2020.

[86] G. Tyukin, G. J.-S. Dovonon, J. Kaddour, and P. Minervini, “Attention is all you need but you don’t need all of it for inference of large language models,” 2024. [Online]. Available: https://arxiv.org/abs/2407.15516

[87] E. Voita, D. Talbot, F. Moiseev, R. Sennrich, and I. Titov, “Analyzing multi-head self-attention: Specialized heads do the heavy lifting, the rest can be pruned,” 2019. [Online]. Available: https://arxiv.org/abs/1905.09418

[88] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le,

and D. Zhou, “Chain-of-thought prompting elicits reasoning in large language models,” 2023. [Online]. Available: https://arxiv.org/abs/2201.11903

[89] G. Wu and H.-I. Suk, “Deep learning in medical image analysis,” *Annual review of biomedical engineering*, vol. 19, 03 2017.

[90] G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, and S. Han, “Smoothquant: Accurate and efficient post-training quantization for large language models,” 2024. [Online]. Available: https://arxiv.org/abs/2211.10438

[91] L. Xu, H. Xie, S.-Z. J. Qin, X. Tao, and F. L. Wang, “Parameter-efficient fine-tuning methods for pretrained language models: A critical review and assessment,” 2023. [Online]. Available: https://arxiv.org/abs/2312.12148

[92] L. Yin, Y. Wu, Z. Zhang, C.-Y. Hsieh, Y. Wang, Y. Jia, G. Li, A. Jaiswal, M. Pechenizkiy, Y. Liang, M. Bendersky, Z. Wang, and S. Liu, “Outlier weighed layerwise sparsity (owl): A missing secret sauce for pruning llms to high sparsity,” 2024. [Online]. Available: https://arxiv.org/abs/2310.05175

[93] M. Yu, D. Wang, Q. Shan, C. Reed, and A. Wan, “The super weight in large language models,” 2024. [Online]. Available: https://arxiv.org/abs/2411.07191

[94] R. Zellers, A. Holtzman, Y. Bisk, A. Farhadi, and Y. Choi, “Hellaswag: Can a machine really finish your sentence?” 2019. [Online]. Available: https://arxiv.org/abs/1905.07830

[95] C. Zhang, S. Bengio, and Y. Singer, “Are all layers created equal?” 2022. [Online]. Available: https://arxiv.org/abs/1902.01996

[96] Q. Zhang, M. Chen, A. Bukharin, N. Karampatziakis, P. He, Y. Cheng, W. Chen, and T. Zhao, “Adalora: Adaptive budget

allocation for parameter-efficient fine-tuning," 2023. [Online]. Available: https://arxiv.org/abs/2303.10512

[97] Y. Zhang, P. Li, J. Hong, J. Li, Y. Zhang, W. Zheng, P.-Y. Chen, J. D. Lee, W. Yin, M. Hong, Z. Wang, S. Liu, and T. Chen, "Revisiting zeroth-order optimization for memory-efficient llm fine-tuning: A benchmark," 2024. [Online]. Available: https://arxiv.org/abs/2402.11592

[98] A. Zhou, Y. Ma, J. Zhu, J. Liu, Z. Zhang, K. Yuan, W. Sun, and H. Li, "Learning n:m fine-grained structured sparse neural networks from scratch," 2021. [Online]. Available: https://arxiv.org/abs/2102.04010

[99] ——, "Learning n:m fine-grained structured sparse neural networks from scratch," 2021. [Online]. Available: https://arxiv.org/abs/2102.04010

[100] Q. Zhou, K. Sheng, X. Zheng, K. Li, X. Sun, Y. Tian, J. Chen, and R. Ji, "Training-free transformer architecture search," 2022. [Online]. Available: https://arxiv.org/abs/2203.12217